



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich



A Systematic Approach to Robust Fast Token Delivery

Master's Thesis

Yuan Qin

yuaqin@student.ethz.ch

Digital Systems and Design Automation (DYNAMO)
Computer Engineering and Networks Laboratory
ETH Zürich

Supervisors:

Prof. Lana Josipović
Prof. Paolo Ienne
Ayatallah Elakhras

April 17, 2026

Abstract

Dynamically scheduled high-level synthesis (HLS) produces elastic circuits in which every piece of data is accompanied by handshake signals and operations fire when and only when all their inputs are available and their downstream consumers are ready, without a predefined schedule. This approach adapts to the irregularity of control flow at runtime, exploiting parallelism that statically scheduled HLS cannot capture. Nevertheless, the direct conversion of a compiler’s intermediate representation into elastic circuits produces circuits with redundant components and limited parallelism. Fast Token Delivery addresses these limitations by delivering tokens directly from producers to consumers, bypassing intermediate basic blocks, and inserting suppression and regeneration circuits to maintain the balance between token production and consumption. However, the suppression circuit itself requires condition tokens that must be delivered to it, creating internal deliveries that the existing algorithm does not fully resolve.

This thesis specifies a complete Fast Token Delivery procedure and develops a suppression algorithm, the core component of the procedure, that handles all internal deliveries during construction. The algorithm is implemented within Dynamatic, an open-source compiler for elastic circuits, and passes all 99 integration tests of the Dynamatic test suite. Evaluation on eleven benchmarks confirms the effectiveness of Fast Token Delivery, with execution time reductions of up to 49% and LUT reductions of up to 32% compared to the baseline circuit generation strategy.

Contents

Abstract	i
1 Introduction	1
2 Background	3
2.1 Dynamically Scheduled Dataflow Circuits	3
2.2 Control Flow and Data Flow Analysis	5
2.3 The Evolution of Token Delivery	9
2.4 Binary Decision Diagrams	14
3 Challenges in Existing Fast Token Delivery	17
3.1 Suppression Introduces New Deliveries	17
3.2 Limitations of the Existing Procedure	18
4 Overall Fast Token Delivery Procedure	20
4.1 GSA Conversion	20
4.2 Regeneration	22
4.3 Suppression	24
4.4 Condition Variable Placeholders	25
4.5 Algorithm Overview	25
5 The Suppression Algorithm	27
5.1 Extracting the Producer-Consumer Relationship	27
5.2 The Decision Graph	33
5.3 Decomposing Cyclic Decision Graphs	36
5.4 From ROBDD to Circuit	40
5.5 Token Distribution	43
5.6 Token Demotion	45
5.7 Conditional Consumption	49
6 Experimental Results	54
7 Conclusion	56
Bibliography	57

Introduction

High-level synthesis (HLS) is the process of automatically generating hardware circuits from programs written in high-level languages such as C or C++. This approach lowers the barrier for adopting FPGAs: software developers without deep hardware expertise can obtain accelerators, and hardware engineers can rapidly prototype designs before committing to a full ASIC flow. HLS tools almost universally rely on building datapaths that are controlled following *static schedules*: operations are translated into hardware components forming a datapath, while a centralized controller determines, at compile time, the cycle when every operation is executed [1]. This approach is inherently conservative. The timing of each operation is fixed without knowledge of the runtime conditions, and any variability in control flow or memory latency must be accounted for pessimistically.

In recent years, a fundamentally different approach has emerged: *dynamically scheduled* circuits, also known as *dataflow* or *elastic* circuits [1, 2, 3]. In this paradigm, the centralized controller is replaced by a distributed coordination mechanism based on handshake signals. Each hardware component communicates with its neighbors through a *valid/ready* protocol: the producer asserts a *valid* signal when a data *token* is available, and the consumer asserts a *ready* signal when it can accept new data. An operation fires (that is, begins execution) when and only when all its input tokens are available and its downstream consumer is ready, without any predefined schedule. This latency-insensitive approach allows the circuit to naturally adapt to runtime conditions, yielding significant performance gains for control-dominated and irregular applications, at the cost of increased area utilization [1, 3].

The foundational work on modern dynamic HLS was presented by Josipović et al. [1], which formed the basis of Dynamatic [4], an open-source C-to-dataflow-circuits compiler. Their strategy performs a *literal translation* of the compiler’s intermediate representation into elastic circuits. Each basic block (BB) in the control flow graph (CFG) is converted into a subcircuit, and tokens are propagated from one BB to the next strictly following the original control flow. Every live-out value of a BB is routed through a **BRANCH** component to the appropriate successor, and every live-in value enters a BB through a **MERGE** component. An in-order control network ensures that BBs are triggered one at a time and in the order dictated by the original program [1, 3, 5].

While this approach is correct, it is also highly conservative. The strict coupling between data propagation and control flow means that tokens must traverse every intermediate BB on their path from producer to consumer, even when the intervening BBs have no use for

the data. This results in circuits with a large number of redundant steering components that increase area and lengthen critical paths. More critically, the in-order control network limits the parallelism that the circuit can exploit. Operations that have no true data dependence on each other are nonetheless serialized because they reside in different BBs that must be triggered sequentially.

To address these limitations, Elakhras et al. [6] proposed a new circuit generation strategy called *Fast Token Delivery*. The core idea is to deliver every token as directly as possible from its producer to its consumer, bypassing intermediate BBs that are irrelevant to the data dependency. This is achieved through analysis of the control dependence graph (CDG) [7] and the adoption of the *Gated Single Assignment* (GSA) form [8, 9], which extends the classical Static Single Assignment (SSA) form with explicit control predicates. By analyzing the control dependencies of producer and consumer BBs, the method determines exactly when a token needs to be *suppressed* (because the producer fires but the consumer will not execute) or *regenerated* (because the producer emits a single token but the consumer, being inside a loop, requires one copy per iteration). The results reported in [6] demonstrate significant improvements in execution time, area, and critical path delay across a range of benchmarks.

However, while the regeneration and suppression mechanisms of [6] resolve the balance between token production and consumption for the original data delivery, they leave a further problem unaddressed, particularly under complex control structures. Suppression relies on a Boolean expression implemented as a circuit. The condition variables in this expression must be delivered to the circuit as tokens, creating new deliveries. These new deliveries cannot be resolved by the same regeneration or suppression procedure, and the original work does not provide a concrete solution.

Contributions. This thesis addresses the above gap. The key contributions are as follows:

1. We specify a complete procedure that converts a CFG in SSA form into an elastic dataflow circuit, covering GSA conversion, regeneration, and suppression.
2. We develop a suppression algorithm that constructs, for any producer-consumer pair, a circuit that correctly discards unneeded tokens. The algorithm resolves all internal deliveries during construction, so that the resulting circuit requires no further processing.
3. We implement the algorithm within Dynamatic [4] and verify its correctness on 99 integration tests. Evaluation on eleven benchmarks confirms that the implementation preserves the performance and area advantages of Fast Token Delivery.

Outline. The remainder of this thesis is organized as follows. Chapter 2 provides the necessary background on dataflow circuits, program analysis, the evolution of token delivery strategies, and binary decision diagrams. Chapter 3 identifies the limitations of the existing Fast Token Delivery algorithm through a concrete example. Chapter 4 specifies the overall procedure, from a CFG in SSA form to an elastic dataflow circuit, and defines the suppression problem that the rest of the thesis addresses. Chapter 5 develops the suppression algorithm in detail. Chapter 6 presents the experimental evaluation. Chapter 7 concludes the thesis.

Background

This chapter introduces the concepts and terminology that are prerequisite to the contributions of this thesis. Section 2.1 describes the principles of dynamically scheduled dataflow circuits. Section 2.2 reviews the relevant control flow and data flow analyses. Section 2.3 traces the evolution of token delivery strategies. Section 2.4 provides an introduction to Binary Decision Diagrams.

2.1 Dynamically Scheduled Dataflow Circuits

The foundational mechanism of dynamically scheduled circuits is the *elastic protocol*, a handshake-based communication scheme between hardware components [1, 3, 6, 10]. As illustrated in Figure 2.1a, a producer component indicates token availability with a 1-bit *valid* signal, and a consumer component indicates its readiness to receive with a 1-bit *ready* signal. A data transfer, called a *token*, occurs when and only when both signals are asserted. If the producer is valid but the consumer is not ready, the token *stalls*. If the consumer is ready but no token is available, the consumer waits. This protocol makes the circuit *latency-insensitive*: the functional correctness of the circuit does not depend on the actual latency of any individual component (e.g., the response time of a memory access), but only on the order of tokens within each channel [10].

Correctness invariants. Two fundamental rules govern the operation of elastic circuits [1, 6]:

1. Every token that is produced must eventually be consumed.
2. Every token must be consumed exactly once.

Together, these two rules ensure a perfect match between the number of tokens produced and the number consumed [6]; we refer to them collectively as the *token-matching* invariants. These invariants are essential for functional correctness. A violation of the first rule leads to deadlock (a token is produced but never consumed, eventually stalling the producer). A violation of the second leads to either data duplication or data loss. Ensuring these invariants across complex control structures is, as we shall see, the central challenge addressed by this thesis.

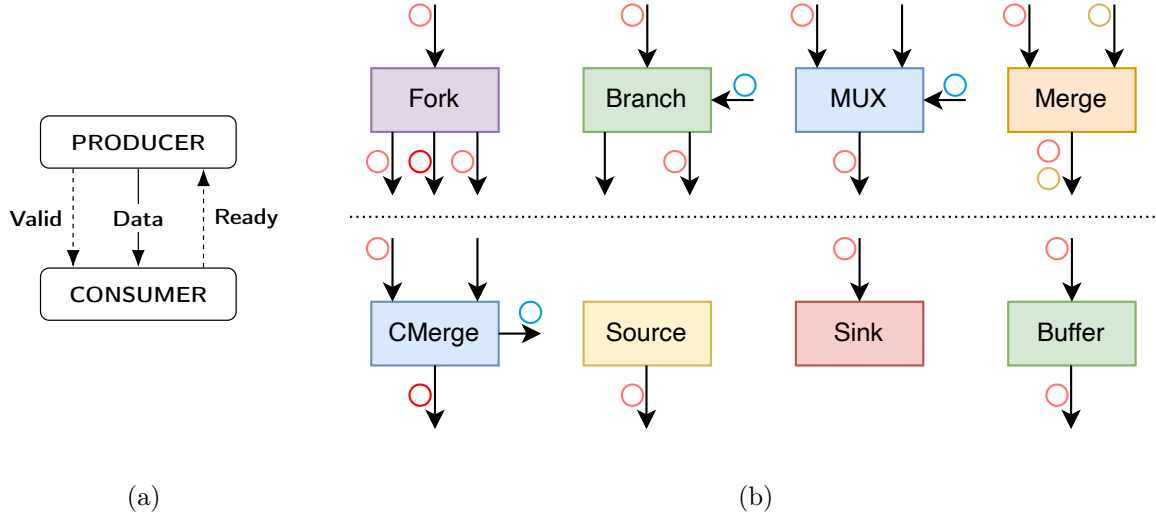


Figure 2.1: (a) The elastic protocol. A data transfer occurs when both the *valid* and *ready* signals are asserted. The token abstracts a *data* signal. (b) Principal elastic components used in dataflow circuits. Each component communicates via the valid/ready handshake protocol.

Core elastic components. Figure 2.1b summarizes the behavior of the principal elastic components used in dataflow circuits. We briefly describe each:

- **Fork (FORK):** Duplicates an incoming token to multiple outputs. A new input token is accepted only after all outputs have been consumed, ensuring the token matching invariant.
- **Branch (BRANCH):** Routes a data token to one of two outputs based on a Boolean condition token. The condition token is always consumed; the data token is forwarded to either the true or the false output.
- **Mux (MUX):** A deterministic merge component that selects one of its data inputs based on a control (select) token. It always consumes a token on the select input and one on the selected data input, but leaves a token waiting on the deselected input if one is available [6].
- **Merge (MERGE):** Forwards a token from any available input to its output. Unlike MUX, MERGE is nondeterministic: the output depends on the arrival times of the inputs [1].
- **Control Merge (CMERGE):** Combines the functionality of MERGE with an additional output indicating which input was selected.
- **Source and Sink:** SOURCE is always valid, providing an endless supply of tokens (e.g., for constants). SINK is always ready, consuming and discarding tokens that are no longer needed.

- **Buffer** (BUFFER): Stores one or more tokens to break combinational cycles (ensuring synchronous correctness) and to absorb backpressure (improving throughput). Buffers can be *transparent* (zero-latency pass-through when no backpressure exists) or *opaque* (always introducing one cycle of latency) [11].

Backpressure and buffer placement. A common issue in dataflow circuits is *backpressure*: when a downstream component is not ready, the “not ready” signal propagates upstream, stalling all preceding components. This effect is particularly acute at FORK components, where one slow successor can block all others. Strategic placement of buffers mitigates this problem by decoupling producers from consumers. Josipović et al. [11] formulated buffer placement and sizing as an optimization problem, and the resulting methodology is integrated into the Dynamatic tool [4]. Since the correctness and performance of the circuits produced by our algorithm depend on appropriate buffering, we rely on this existing infrastructure and do not address buffer placement further in this thesis.

2.2 Control Flow and Data Flow Analysis

The Fast Token Delivery methodology relies on several program analysis concepts from compiler theory: the control flow graph and its loop structure, dominance and post-dominance relations, control dependence, and the SSA and GSA intermediate representations. This section defines each of them.

Control Flow Graph

A *basic block* (BB) is a maximal-length sequence of instructions that can only be entered at its first instruction and contains no branches except possibly at its last instruction. A *control flow graph* (CFG) is a directed graph $G = (V, E)$, where V is the set of basic blocks and $E \subseteq V \times V$ is the set of directed edges representing possible transfers of control [12]. Each basic block $B \in V$ is assigned a unique integer index $\text{idx}(B) \in \mathbb{N}$. Every CFG has a unique *entry node* (without predecessors) and a unique *exit node* (without successors).

Each basic block has at most two successors:

$$\text{succ}(B) = \{ S \mid (B \rightarrow S) \in E \}, \quad 0 \leq |\text{succ}(B)| \leq 2.$$

Blocks with 0, 1, and 2 successors correspond to terminal, sequential, and conditional blocks, respectively. A terminal block is the exit node. A sequential block ends with an unconditional branch. A conditional block ends with a conditional branch whose two outgoing edges correspond to the true and false outcomes.

The CFG is assumed to be *topologically sorted* except for back edges: for every non-back edge $(B_i \rightarrow B_j) \in E$, $\text{idx}(B_i) < \text{idx}(B_j)$. A directed edge $(B \rightarrow S) \in E$ is a *back edge* if and only if $\text{idx}(S) < \text{idx}(B)$. Back edges are the only edges that violate the topological order and identify loops. The target of a back edge is the *loop header*, and the set of nodes reachable from the header without leaving the strongly connected subgraph form the *loop body*.

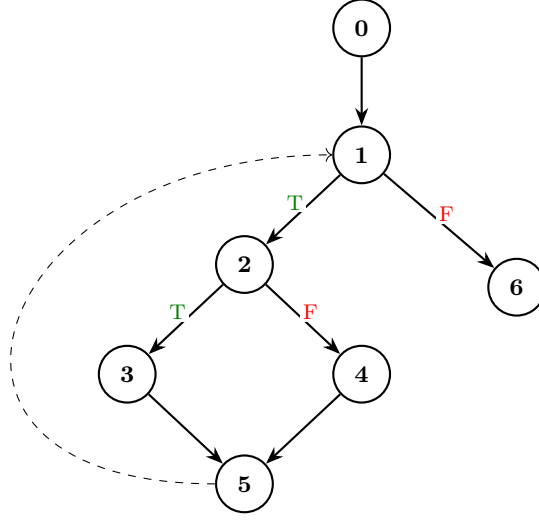


Figure 2.2: An example CFG with a loop. The dashed edge is the backward edge. T and F denote true and false branches.

Figure 2.2 shows an example CFG with seven basic blocks. BB0 is the entry node and transfers control unconditionally to BB1. BB1 has two successors: BB2 when the loop condition is true, and BB6 when it is false. BB2 also has two successors: BB3 and BB4, depending on the value of a conditional. BB3 and BB4 each have one successor, BB5, which transfers control back to BB1. The edge from BB5 to BB1 is a back edge. BB1 is the loop header, and {BB1, BB2, BB3, BB4, BB5} form the loop body. BB6 is the exit node.

Dominance and Post-Dominance

A node v *dominates* w (written $v \text{ dom } w$) if every path from the entry node to w passes through v . Following the convention of Cytron et al. [13], dominance is *reflexive*: every node dominates itself ($v \text{ dom } v$ for all v). A node v *strictly dominates* w (written $v \text{ sdom } w$) if $v \text{ dom } w$ and $v \neq w$. The *immediate dominator* of w , denoted $\text{idom}(w)$, is the unique node v such that $v \text{ sdom } w$ and every other strict dominator of w also strictly dominates v . Every node except the entry node has a unique immediate dominator, and the immediate-dominator relation induces a tree rooted at the entry node, called the *dominator tree* [12].

Dually, u *post-dominates* w (written $u \text{ pdom } w$) if every path from w to the exit node passes through u . Post-dominance is also reflexive. *Strict post-dominance*, *immediate post-dominator*, and the *post-dominator tree* (rooted at the exit node) are defined analogously.

Figure 2.3 shows both trees for the CFG of Figure 2.2. In the dominator tree, BB2 dominates BB3, BB4, and BB5 because they are only reachable through BB2. BB6 is dominated by BB1, because BB6 is reached directly from BB1 when the loop exits. In the post-dominator tree, BB5 post-dominates BB2, BB3, and BB4 because both branches of the conditional converge at BB5.

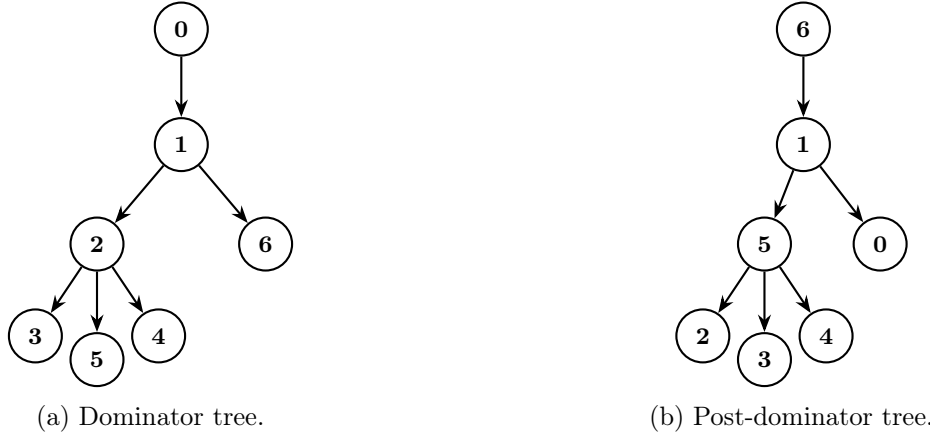


Figure 2.3: Dominator and post-dominator trees for the CFG in Figure 2.2.

Control Dependence

Dominance and post-dominance capture the structure of the CFG. *Control dependence*, introduced by Ferrante et al. [7], captures a different relation: which branch decisions determine whether a given block executes.

We adopt the formulation of Cytron et al. [13], which is consistent with the reflexive dominance convention introduced above. A node Y is *control dependent* on a node X if the following two conditions hold:

1. There exists a nonnull path $p: X \rightarrow \dots \rightarrow Y$ such that Y post-dominates every node after X on p .
2. Y does not strictly post-dominate X .

In other words, there is some edge leaving X that guarantees the execution of Y , and there is also some path from X that avoids Y . We label each such dependence with the branch outcome (true or false) at X that causes Y to execute.

The *control dependence graph* (CDG) has the same node set as the CFG plus an additional entry node S representing program start, with an edge from X to Y whenever Y is control dependent on X . Blocks that execute unconditionally, such as the CFG entry and exit, are control dependent on S . Figure 2.4 shows the CDG for the CFG of Figure 2.2. BB2 and BB5 are control dependent on BB1 (the loop condition): they execute only when BB1 takes the true branch. BB3 is control dependent on BB2 with the true label, and BB4 with the false label. BB1 is control dependent on itself, reflecting the fact that a new loop iteration begins only when BB1 takes its true branch.

In the Fast Token Delivery methodology [6], control dependence plays a central role: given a producer in some block B_p and a consumer in some block B_c , the set of blocks on which B_p and B_c are control dependent identifies exactly the branch decisions that determine whether each operation will fire.

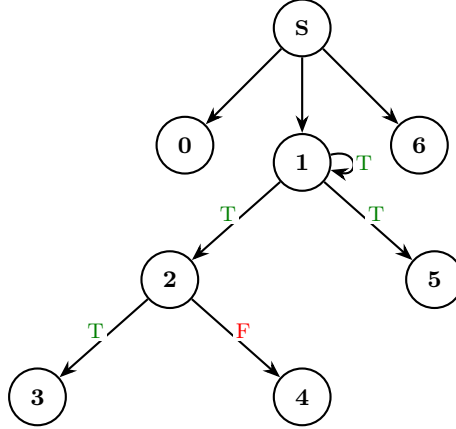


Figure 2.4: Control dependence graph for the CFG in Figure 2.2. Node S represents the program entry. Labels indicate which branch outcome at the source causes the target to execute.

Transitive closure. The control dependence relation can be extended to its *transitive closure*: a block Z is *transitively control dependent* on X if there exists a chain of blocks $X, Y_1, Y_2, \dots, Z$ such that each block in the chain is control dependent on its predecessor. For example, in Figure 2.4, BB3 is directly control dependent on BB2, and BB2 is control dependent on BB1. The transitive closure therefore includes the pair (BB1, BB3): the execution of BB3 depends on branch decisions at both BB1 and BB2. Computing the full set of branch conditions affecting a given block requires this transitive closure, and it is precisely these conditions that determine the suppression and regeneration logic in Fast Token Delivery.

Forward control dependence. Control dependence as defined above is computed on the full CFG, including back edges. For certain analyses, it is useful to consider only the acyclic portion of the CFG. *Forward control dependence* is obtained by removing all back edges from the CFG and computing control dependence on the resulting directed acyclic graph (DAG).

Static Single Assignment

The *Static Single Assignment* (SSA) form [13] is a naming discipline for compiler intermediate representations in which every variable is assigned exactly once. When a variable may receive different values depending on the control flow path (for instance, at the confluence of an if-then-else construct), a special ϕ -function is inserted to merge the reaching definitions into a single name. SSA simplifies many compiler analyses because every use of a variable has a single, unambiguous reaching definition.

However, ϕ -functions are inherently *nondeterministic*: they select the correct value based on which control flow edge was taken, but this information is not explicitly encoded in the function itself. In the context of elastic circuits, this nondeterminism is problematic because a ϕ -function translates naturally into a **MERGE** component [1], which selects inputs based on

arrival order rather than on the intended program semantics.

Gated Single Assignment

The *Gated Single Assignment* (GSA) form, introduced by Ballance et al. [8] and refined by Campbell et al. [9], resolves the limitation of SSA by replacing nondeterministic ϕ -functions with *gating functions* that carry explicit control predicates. GSA defines two principal types of gates:

- γ (**gamma**): Used at if-then-else confluences. A γ -function takes three arguments, a predicate P , a true-value v_{true} , and a false-value v_{false} , and returns v_{true} when P is true, v_{false} otherwise. In hardware, a γ -function maps directly to a MUX with an explicit select signal derived from P .
- μ (**mu**): Used at loop headers where a value may come from outside the loop (on the first iteration) or from within the loop (on subsequent iterations). A μ -function takes a predicate P , an initialization value v_0 , and an iterative value v_{iter} . On the first iteration, v_0 is selected; thereafter, v_{iter} is used until the loop terminates.

By making the control predicates explicit, GSA provides complete information about which value should be selected and under what conditions, without requiring any reference to the control flow graph at runtime.

2.3 The Evolution of Token Delivery

This section describes how tokens are delivered from producers to consumers in elastic circuits, tracing the evolution from the baseline strategy to the Fast Token Delivery methodology.

Literal Translation of the Control Flow Graph

The baseline strategy, introduced by Josipović et al. [1], performs a *literal translation* of the compiler’s data and control flow graphs into an elastic circuit. The conversion proceeds as follows.

First, a datapath is created for each BB: every operator corresponds to a functional unit, and edges in the data flow graph become connections between components. A **MERGE** node is allocated for every variable entering a BB (live-in), receiving data from one of the predecessor BBs. For every value used by a successor BB (live-out), a **BRANCH** is placed at the BB output to route the token to the correct successor.

A key requirement is that data propagation must be strictly coupled with control flow. The data and control flow graphs are modified to ensure that every BB provides live-out values only to its immediate CFG successors, and receives live-in values only from its immediate CFG predecessors. This guarantees that every piece of data receives a fresh token each time the BB actually executes.

In addition to the datapath, the circuit contains a network of dataless components that carries a single control token propagating strictly along the control flow of the program [6].

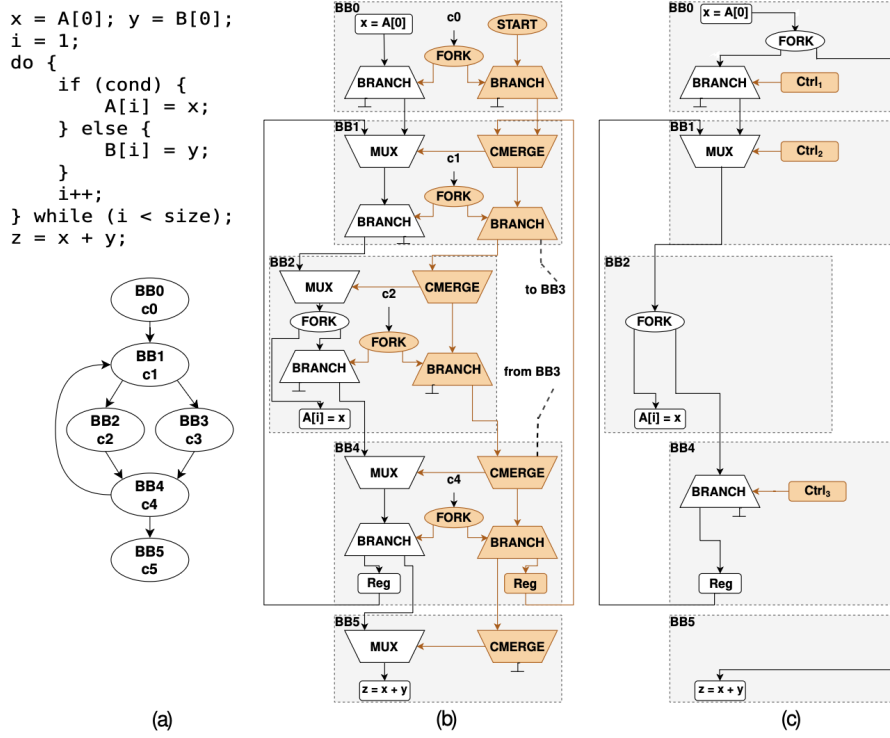


Figure 2.5: Contrasting circuit generation strategies for a simple loop kernel. (a) The source code and its CFG. (b) The literal translation [1] forces tokens to traverse every BB in program order, producing a circuit with many redundant steering components and limited parallelism. (c) Fast Token Delivery [6] delivers tokens directly from producers to consumers, yielding a simpler circuit that exposes more parallelism. Reproduced from [6].

At each BB, a CMERGE component receives this token from one of the predecessor BBs and sends the information of the input origin to all MUX components in the same BB, indicating which input token they must consume next.

Figure 2.5b illustrates the circuit obtained by this strategy for the code and its CFG in Figure 2.5a. The orange components form the control network; the remaining components form the datapath. As observed in [6], this strict adherence to the CFG leads to two problems. First, the circuit contains many redundant components: tokens for a variable must pass through every intermediate BB on their way from producer to consumer, even when those BBs have no use for the data. Second, the control network forces BBs to trigger one at a time and in order, limiting the parallelism between different operations.

Fast Token Delivery

Elakhras et al. [6] proposed *Fast Token Delivery* to overcome the limitations of the literal translation. The key insight is that tokens should be delivered *as directly as possible* from producers to consumers, bypassing intermediate BBs that play no role in the control depen-

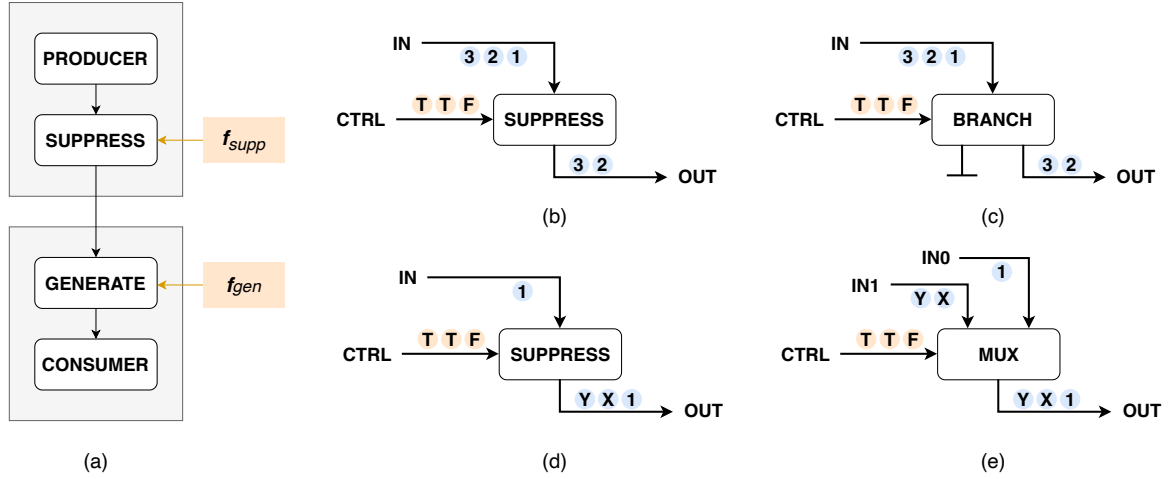


Figure 2.6: The generic delivery path between a producer and a consumer, adapted from [6]. (a) A SUPPRESS and a GENERATE component are inserted between producer and consumer. (b) SUPPRESS is implemented as a BRANCH with a SINK: tokens are discarded when f_{supp} is true. (c) GENERATE is implemented as a MUX with a SOURCE: dummy tokens are injected when f_{gen} is true.

dency.

Core mechanism. For each producer–consumer pair, the methodology connects the producer to the consumer through two control components, as shown in Figure 2.6a. A SUPPRESS component is placed after the producer, and a GENERATE component is placed before the consumer.

- The SUPPRESS component (Figure 2.6b) eliminates a token when a Boolean control signal f_{supp} evaluates to true. Suppression is needed when the producer fires but the consumer will not execute. It is implemented as a BRANCH (Figure 2.6c) whose true output is connected to a SINK: when f_{supp} is true, the token is discarded; when false, it passes through to the consumer.
- The GENERATE component (Figure 2.6d) injects a dummy token when a Boolean control signal f_{gen} evaluates to true. Generation is needed when the consumer requires a token that the producer has not produced. It is implemented as a MUX (Figure 2.6e) with one input connected to a SOURCE: when f_{gen} is true, a fresh token is inserted; when false, the original token passes through. For data paths, f_{gen} is always false (consuming an unproduced data value is nonsensical); generation may be required only on control paths.

Computing delivery conditions. The conditions f_{supp} and f_{gen} are derived from the CDG through a five-step procedure. We illustrate each step on the CFG in Figure 2.7a,

where variable x has two producers (in BB2 and BB3) and one consumer (in BB4). The corresponding CDG is shown in Figure 2.7b.

Step 1: Identify control dependencies. Using the CDG, compute the sets S_{prod} and S_{cons} , which are the transitive closures of the control dependencies of the producer BB and the consumer BB, respectively. For the producer in BB2: $S_{\text{prod}} = \{\text{BB0}\}$. For the consumer in BB4: $S_{\text{cons}} = \{\text{BB0}, \text{BB2}, \text{BB3}\}$.

Step 2: Eliminate common control ancestors. Remove $S_{\text{prod}} \cap S_{\text{cons}}$ from both sets. These common ancestors affect producer and consumer equally and play no role in the token mismatch. In our example, BB0 is common, so $S_{\text{prod}} = \emptyset$ and $S_{\text{cons}} = \{\text{BB2}, \text{BB3}\}$.

Step 3: Compute conditions of production and consumption. Derive Boolean expressions f_{prod} and f_{cons} by traversing the CDG from each BB in the respective sets to the producer/consumer BB. Each path yields a product of elementary branch conditions, and the sum of all such products gives the expression. An empty set yields a constant true expression. In our example: $f_{\text{prod}} = 1$ and $f_{\text{cons}} = c_2 + \overline{c_2} \cdot c_3$ (BB4 is reached either through the true branch of BB2, or through the false branch of BB2 followed by the true branch of BB3).

Step 4: Adjust for MUX consumers. If the consumer is a MUX (as it is here, since two producers deliver to the same variable in BB4), the consumption condition must account for the select signal f_{sel} of the MUX. The condition is adjusted by multiplying f_{cons} with f_{sel} or its complement, so that the consumption condition is true only when the MUX input connected to the current producer is active. For the producer in BB2, $f_{\text{sel}} = c_2$, giving $f_{\text{cons}} = c_2 \cdot (c_2 + \overline{c_2} \cdot c_3) = c_2$. For the producer in BB3, no adjustment is needed because BB3 is itself control dependent on BB2 (whose condition is the select signal), so $f_{\text{cons}} = c_3$.

Step 5: Compute suppression and generation conditions. $f_{\text{supp}} = f_{\text{prod}} \cdot \overline{f_{\text{cons}}}$ and $f_{\text{gen}} = \overline{f_{\text{prod}}} \cdot f_{\text{cons}}$. For the producer in BB2: $f_{\text{supp}} = 1 \cdot \overline{c_2} = \overline{c_2}$ and $f_{\text{gen}} = 0$. For the producer in BB3: $f_{\text{supp}} = 1 \cdot \overline{c_3} = \overline{c_3}$ and $f_{\text{gen}} = 0$. The resulting delivery circuit is shown in Figure 2.7(c): each producer path has a SUPPRESS controlled by f_{supp} , and a MUX at BB4 selects between the two inputs using f_{sel} .

Delivering control tokens. A subtle difficulty arises when the computed conditions f_{supp} or f_{gen} are themselves functions of multiple branch conditions, some of which may not always produce a token. Consider a condition $f_{\text{supp}} = \overline{c_0} \cdot \overline{c_1}$, where c_1 is only generated when c_0 is false. If c_0 is true, the token for c_1 will never arrive, and a naïve implementation would deadlock. The solution proposed in [6] is to implement the Boolean function using a sequence of Shannon expansions ordered by control dependence (Algorithm 2 in [6]): literals are ordered so that each literal’s availability depends only on preceding literals, and the MUX-based implementation avoids consuming tokens on deselected inputs, sidestepping the missing-token problem.

Figure 2.5c shows the circuit obtained by Fast Token Delivery. Compared to the literal translation, the circuit is dramatically simpler: the variable x is delivered directly to the adder $z = x + y$ without traversing the loop, and the computation of z can begin almost immediately after the circuit starts, well before the loop exits.

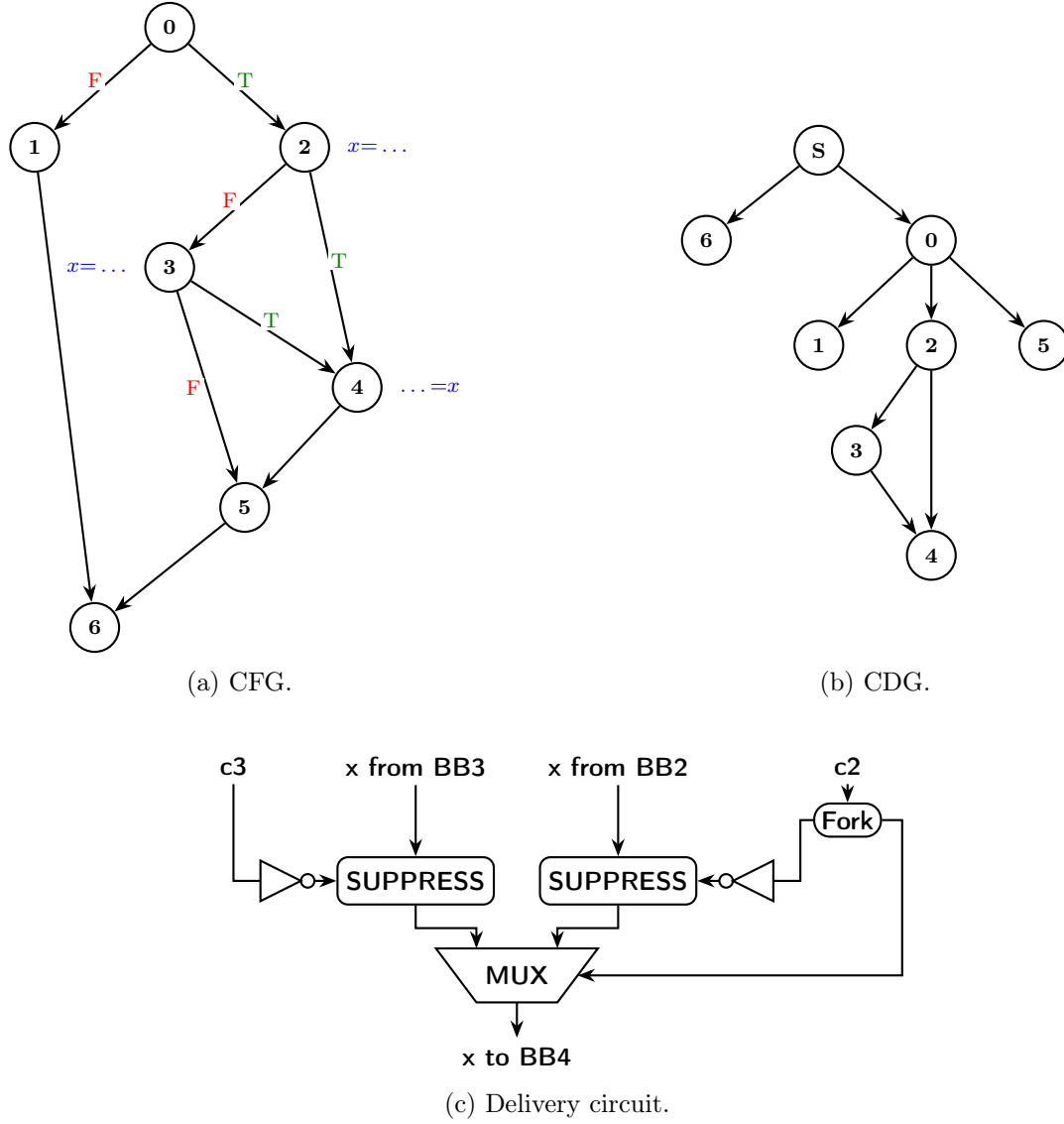


Figure 2.7: Delivery of variable x from two producers (BB2, BB3) to one consumer (BB4). (a) CFG. (b) The corresponding CDG. (c) The delivery circuit.

2.4 Binary Decision Diagrams

As shown in the preceding section, the delivery conditions f_{supp} and f_{gen} are Boolean functions of branch conditions, and their circuit implementation relies on Shannon expansion into MUX-trees. This section introduces the formal framework for representing and manipulating such Boolean functions.

Definition

Following Bryant [14], a *binary decision diagram* (BDD) is a rooted directed graph

$$D = (U, A, r),$$

where U is the set of vertices, $A \subseteq U \times U$ is the set of directed edges, and $r \in U$ is the distinguished root. Every vertex in U is reachable from the root r .

The vertex set U is partitioned into *nonterminal* and *terminal* vertices:

$$U = U_{\text{nt}} \uplus U_{\text{t}}.$$

Each nonterminal vertex $u \in U_{\text{nt}}$ is labeled with a Boolean variable $x_{\text{index}(u)}$, where

$$\text{index}(u) \in \{1, \dots, n\}$$

identifies the variable being tested at u . The vertex has two children: $\text{low}(u)$, reached when $x_{\text{index}(u)} = 0$ (false), and $\text{high}(u)$, reached when $x_{\text{index}(u)} = 1$ (true). Edges are induced by children:

$$(u, \text{low}(u)) \in A \text{ and } (u, \text{high}(u)) \in A \quad \text{for all } u \in U_{\text{nt}}.$$

Each terminal vertex $u \in U_{\text{t}}$ has $\text{value}(u) \in \{0, 1\}$, representing the function output.

The function value for a given input assignment is obtained by traversing the graph from the root, following the **low** or **high** edge at each nonterminal vertex according to the corresponding variable's value, until a terminal vertex is reached. The **value** of that terminal is the function output.

Ordered and Reduced Properties

The BDD is *ordered* (property O) if variable indices strictly increase along edges to nonterminal children:

$$\forall u \in U_{\text{nt}} : \text{low}(u) \in U_{\text{nt}} \Rightarrow \text{index}(u) < \text{index}(\text{low}(u)),$$

$$\forall u \in U_{\text{nt}} : \text{high}(u) \in U_{\text{nt}} \Rightarrow \text{index}(u) < \text{index}(\text{high}(u)).$$

Along any path from root to terminal, the encountered indices are strictly increasing. In particular, this implies acyclicity.

An ordered BDD is *reduced* (property R) if it satisfies two conditions:

1. No nonterminal vertex u has $\text{low}(u) = \text{high}(u)$ (no redundant tests).

2. No two distinct vertices u and v root isomorphic subgraphs (no duplicate subgraphs).

Two BDDs $G = (U, A, r)$ and $G' = (U', A', r')$ are *isomorphic* if there exists a bijection $\sigma : U \rightarrow U'$ such that for every vertex $u \in U$: if u and $\sigma(u)$ are terminal, then $\text{value}(u) = \text{value}(\sigma(u))$; if nonterminal, then $\text{index}(u) = \text{index}(\sigma(u))$, $\sigma(\text{low}(u)) = \text{low}(\sigma(u))$, and $\sigma(\text{high}(u)) = \text{high}(\sigma(u))$.

ROBDD and Canonicity

A *Reduced Ordered Binary Decision Diagram* (ROBDD) is a BDD satisfying both properties O and R. Bryant [14] and Brace et al. [15] established that ROBDDs provide a *canonical* representation of Boolean functions under a fixed variable ordering: for a given ordering, two Boolean functions are equal if and only if their ROBDDs are isomorphic. In particular, there is a one-to-one correspondence between Boolean functions and ROBDDs.

Structural Properties

Non-post-domination. In an ROBDD, no vertex post-dominates another distinct vertex.

Proof. Let u and v be two distinct vertices. If no directed path from u to v exists, then v trivially does not post-dominate u . Otherwise, among all directed paths from u to v , consider one in which the last vertex before reaching v has the maximal index, and denote this vertex by w . Since the index of w is maximal, the other successor of w cannot reach v . Therefore there exists a path from u that traverses w and follows this alternative successor without passing through v , so v does not post-dominate u . $\square$

Dual terminal reachability. Every nonterminal vertex in an ROBDD has at least one path to each of the two terminal vertices. If a nonterminal vertex could reach only one terminal, that terminal would post-dominate the vertex, contradicting the non-post-domination property.

Shannon Expansion and MUX-Tree Mapping

The Shannon expansion of a Boolean function f with respect to a variable x_i is

$$f = \overline{x_i} \cdot f|_{x_i=0} + x_i \cdot f|_{x_i=1}.$$

This corresponds directly to a MUX operation:

$$f = \text{MUX}(x_i, f|_{x_i=0}, f|_{x_i=1}),$$

where x_i is the select signal, $f|_{x_i=0}$ is the input selected when x_i is false, and $f|_{x_i=1}$ is the input selected when x_i is true.

Each nonterminal vertex in an ROBDD corresponds to one such expansion step: the variable label provides the select signal, and the **low** and **high** children provide the two data inputs. When a child is a terminal vertex, the corresponding MUX input is a constant (0 or 1).

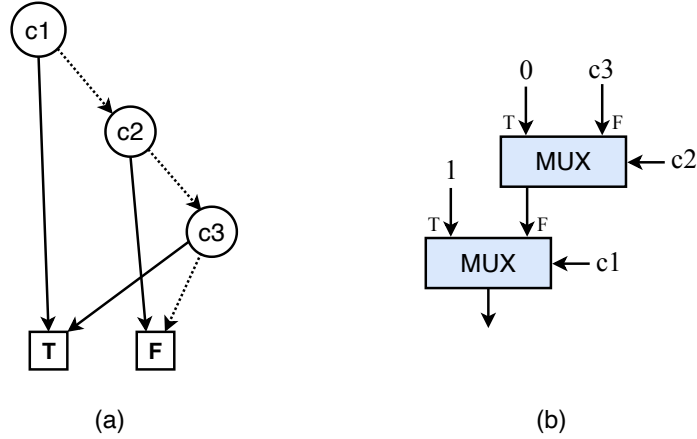


Figure 2.8: Mapping from (a) the ROBDD to (b) the MUX-tree for $f = c_1 + \overline{c_1} \cdot \overline{c_2} \cdot c_3$. Dashed edges denote false edges.

When a child is a nonterminal vertex whose subtree consists of a single variable x_j (i.e., x_j has no further nonterminal descendants beyond terminals), then x_j itself (or its complement) serves directly as a data input rather than a select signal of a further MUX. An ROBDD therefore maps directly to a MUX-tree in hardware. Because the ROBDD is reduced, the resulting MUX-tree contains no redundant multiplexers, and because it is ordered, the select signals appear in a consistent order along every path.

Figure 2.8 illustrates this mapping on the Boolean function $f = c_1 + \overline{c_1} \cdot \overline{c_2} \cdot c_3$. Expanding with respect to c_1 gives $f|_{c_1=1} = 1$ and $f|_{c_1=0} = \overline{c_2} \cdot c_3$. Expanding the cofactor with respect to c_2 gives $f|_{c_2=1} = 0$ and $f|_{c_2=0} = c_3$. The resulting ROBDD has three nonterminal vertices. In the MUX-tree, the root MUX uses c_1 as its select signal with constant 1 on the true input. The second MUX uses c_2 as its select signal with constant 0 on the true input and c_3 directly as a data input on the false side, since c_3 is the last remaining variable before the terminals.

This mapping is the mechanism used by Algorithm 2 in [6] to implement delivery conditions as elastic circuits: the successive Shannon expansions are ordered by control dependence, and the property of MUX not consuming tokens on deselected inputs ensures that missing branch-condition tokens do not cause deadlock.

Since each Boolean function corresponds to a unique ROBDD under a fixed variable ordering [14, 15], any two MUX-trees derived from the same ROBDD are guaranteed to implement the same Boolean function.

Challenges in Existing Fast Token Delivery

The preceding chapter presented the Fast Token Delivery methodology of [6], including the computation of suppression conditions and their implementation as MUX-trees via Shannon expansion. For the examples considered in [6], this procedure produces correct circuits. However, certain control structures cause the suppression circuit itself to violate the token-matching invariants. This chapter presents a concrete example of such a violation and explains why the existing methodology cannot resolve it.

3.1 Suppression Introduces New Deliveries

Consider the CFG in Figure 3.1a. Variable x is produced in BB0 and consumed in BB3. BB0 branches to BB2 (when c_0 is true) or BB1 (when c_0 is false). BB1 branches to BB2 (when c_1 is true) or BB3 (when c_1 is false). BB2 branches to BB4 (when c_2 is true) or BB3 (when c_2 is false). Among all paths from BB0, the consumer is not reached on paths BB0 $\rightarrow$ BB2 $\rightarrow$ BB4 and BB0 $\rightarrow$ BB1 $\rightarrow$ BB2 $\rightarrow$ BB4. Following the procedure of Section 2.3, the suppression condition is

$$f_{\text{supp}} = c_0 \cdot c_2 + \overline{c_0} \cdot c_1 \cdot c_2.$$

The Shannon expansion of f_{supp} produces the MUX-tree in Figure 3.1b. The root MUX has select signal c_0 , true input c_2 , and false input from an inner MUX. The inner MUX has select signal c_1 , true input c_2 , and false input the constant 0. Each MUX input that is not a constant consumes a condition token produced by the corresponding conditional block. For c_0 and c_1 , each token is consumed by exactly one input. For c_2 , two inputs each expect a token, so a FORK produces a copy for each.

The token-matching violation. On any given execution, only one of the two copies of c_2 is consumed. BB2 is reached either directly from BB0 (when c_0 is true) or through BB1 (when c_0 is false and c_1 is true). In the first case, BB1 does not execute and the inner MUX receives no select token, so its copy of c_2 is never consumed. In the second case, the root MUX receives $c_0 = \text{false}$ and passes the evaluation to the inner MUX, leaving the copy of c_2 at

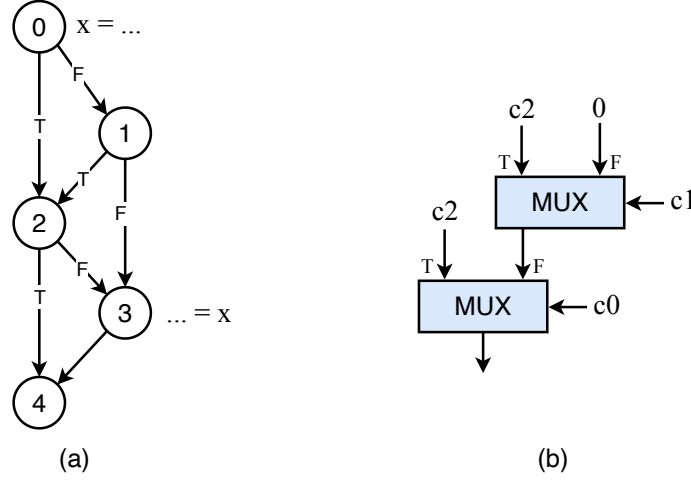


Figure 3.1: (a) CFG with producer in BB0 and consumer in BB3. (b) MUX-tree for $f_{\text{supp}} = c_0 \cdot c_2 + \overline{c_0} \cdot c_1 \cdot c_2$. The variable c_2 appears at two MUX inputs.

its true input unconsumed. In both cases, one copy of c_2 is produced but never consumed, violating the token-matching invariants.

The unused copy must be discarded to avoid stalling the circuit. This means the delivery of c_2 to the MUX-tree inputs requires its own suppression logic. Constructing the suppression circuit for the delivery of x has introduced new deliveries of condition tokens that the methodology of [6] does not address.

3.2 Limitations of the Existing Procedure

In [6], it is suggested that condition token delivery can be handled by applying the delivery procedure recursively. In a normal delivery, the procedure computes suppression based on whether control flow reaches the consumer's block from the producer's block. For the new deliveries within the suppression circuit, the relevant question is fundamentally different.

Consider the c_2 delivery in the example above. Which MUX input should receive c_2 on a given execution depends on the path from BB0 (the original producer of x) to BB2 (where c_2 is produced). As shown in Section 3.1, the two paths correspond to the two MUX inputs, and the delivery of c_2 is governed by c_0 and c_1 , which describe how control reached BB2 from BB0. The delivery procedure of [6] does not answer this kind of question. It determines whether control flow reaches a consumer block, not which path was taken to reach a producer block.

Attempting to apply the procedure anyway leads to concrete failures. For example, assigning the consumer to BB3 (the original consumer block of x) and computing a suppression condition for c_2 would suppress both tokens of c_2 on the path $\text{BB0} \rightarrow \text{BB2} \rightarrow \text{BB4}$, because BB3 is not reached. But the root MUX needs c_2 on its true input precisely on this path. If c_2 is suppressed, the MUX-tree cannot compute f_{supp} , the BRANCH for suppression never receives

its select signal, and the circuit deadlocks. Although the delivery procedure multiplies **MUX** conditions when the consumer is a **MUX** input, which may seem related to this scenario, the two are not the same. To make it concrete, in this example, c_0 and c_1 , the conditions we need, are control dependencies of BB2 and would be filtered out regardless of which block the consumer is assigned to.

The example above illustrates one type of token mismatch that arises in the internal deliveries of a suppression circuit. Depending on the structure of the CFG, other types of mismatch can occur as well. A complete treatment is given in Chapter 5.

Approach of this thesis. The problem identified above indicates that the existing FTD algorithm lacks a precise formulation that guarantees the generated circuits satisfy the token-matching invariants in all cases. This thesis addresses this gap by specifying a complete FTD procedure that is concrete, robust, and guaranteed to terminate. The suppression algorithm resolves all internal token routing during the construction of the suppression circuit, so that after processing a producer-consumer pair, no further handling of newly introduced deliveries is needed. Chapter 4 presents the overall procedure, and Chapter 5 develops the main algorithmic contribution: the construction of circuits for suppression expressions that satisfy the token-matching invariants.

Overall Fast Token Delivery Procedure

This chapter specifies the complete FTD procedure developed in this thesis, from a CFG in SSA form to an elastic dataflow circuit. The procedure begins by converting the SSA representation to GSA form. Since the GSA implementation determines the set of producer-consumer deliveries and their conditions, Section 4.1 describes the specific construction used in this thesis. Following the GSA conversion, operations within each basic block are translated into elastic components, and data-dependent components within each block are connected, as in [6]. The procedure then processes all producer-consumer deliveries across basic blocks in two stages. First, regeneration is performed on all deliveries where the producer is outside a loop and the consumer is inside (Section 4.2). After regeneration, the only remaining delivery problem is that a producer fires but the consumer does not execute, and the procedure constructs a suppression circuit for each such case (Section 4.3). Section 4.4 addresses the delivery of condition variables to their conditional blocks. Section 4.5 gives the complete algorithm.

4.1 GSA Conversion

This section describes the GSA conversion. Every ϕ -function is replaced by a μ -function (at loop headers), a γ -tree (at non-loop confluences), or a μ -function with γ -trees on one or both of its inputs (when a loop header receives multiple definitions from outside or inside the loop). After the conversion, every variable use has a deterministic reaching definition, and the control predicate that selects among multiple definitions is made explicit.

μ -Functions

A ϕ -function is converted to a μ -function if all of the following conditions hold:

1. The ϕ -function has exactly two inputs.
2. The ϕ -function resides in a block that is a loop header.
3. One input originates from outside the loop and the other from inside the loop.

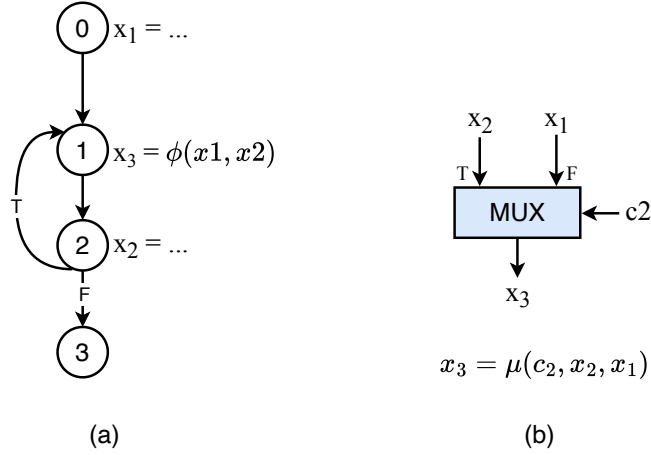


Figure 4.1: Conversion of a ϕ -function to a μ -function. (a) CFG with a loop: x_1 is defined outside the loop in BB0, x_2 inside the loop in BB2, and $\phi(x_1, x_2)$ at the loop header BB1. (b) The μ -function implemented as a MUX with select signal c_2 .

Figure 4.1 illustrates the conversion. In the CFG, BB0 defines x_1 and transfers control to BB1, which is a loop header containing $x_3 = \phi(x_1, x_2)$. BB1 transfers control to BB2, where x_2 is defined. BB2 branches back to BB1 (when c_2 is true) or exits to BB3 (when c_2 is false). The ϕ -function at BB1 satisfies all three conditions, so it is replaced by $x_3 = \mu(c_2, x_2, x_1)$. In hardware, this becomes a MUX whose select input is the loop exit condition c_2 , combined with an INIT component that forces the selection of the external input x_1 on the very first iteration, before c_2 has been computed. Computing the loop exit condition is equivalent to computing the suppression condition from the loop header to itself through the back edge. In this example, the condition is simply c_2 , but in more complex loop structures it becomes a Boolean expression over multiple branch conditions, giving rise to the same issues identified in Chapter 3. The robust construction of a circuit for this expression is developed in Chapter 5.

γ -Trees

When multiple definitions of a variable converge at a ϕ -function that is not a μ -function, the ϕ -function is replaced by a tree of γ -functions. Each γ -function is a two-input MUX with an explicit select signal derived from a branch condition.

The γ -tree is constructed by successive Shannon expansion over the branch conditions that distinguish the definitions, following the approach described in [6]. The expansion proceeds on a reduced version of the CFG: all back edges are removed, and any branch from which the ϕ -function's block is unreachable is pruned. Starting from the common dominator ancestor of all source blocks, the condition variable with the lowest block index is used as the select signal of the first γ -function. The definitions are partitioned into those reached when the condition is true and those reached when it is false, and the expansion continues recursively on each partition. If a partition contains only one definition, that definition is used directly without further expansion.

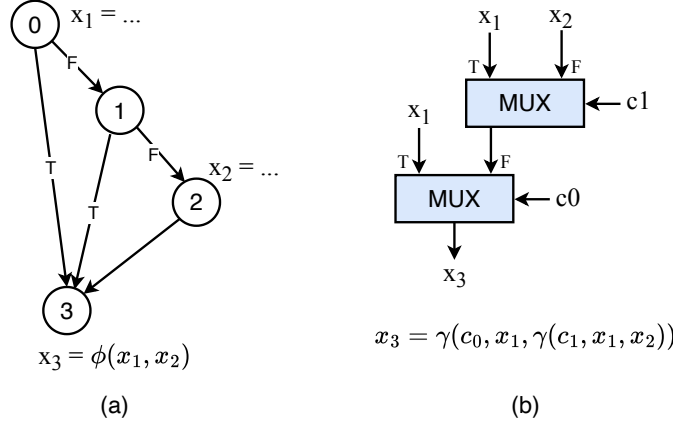


Figure 4.2: Construction of a γ -tree. (a) CFG with x_1 defined in BB0, x_2 defined in BB2, and $\phi(x_1, x_2)$ at BB3. (b) The resulting γ -tree $\gamma(c_0, x_1, \gamma(c_1, x_1, x_2))$.

Figure 4.2 illustrates the construction. In the CFG, x_1 is defined in BB0 and x_2 in BB2, with $x_3 = \phi(x_1, x_2)$ at BB3. BB0 branches to BB3 (when c_0 is true) or BB1 (when c_0 is false). BB1 branches to BB3 (when c_1 is true) or BB2 (when c_1 is false). BB2 transfers control unconditionally to BB3. On the reduced CFG, there are three paths from the common dominator BB0 to BB3: via c_0 (selecting x_1), via $\overline{c_0} \cdot c_1$ (selecting x_1), and via $\overline{c_0} \cdot \overline{c_1}$ (selecting x_2). Expanding on c_0 first: when c_0 is true, only x_1 remains. When c_0 is false, a second γ with select c_1 distinguishes x_1 (true) from x_2 (false). The result is $x_3 = \gamma(c_0, x_1, \gamma(c_1, x_1, x_2))$.

μ with γ -Trees

When a ϕ -function at a loop header has multiple definitions reaching it from outside the loop, from inside the loop, or both, a μ -function is constructed at the header, and γ -trees are attached to its external input, internal input, or both, to select among the multiple definitions within each group.

4.2 Regeneration

After the GSA conversion, every variable use has a single reaching definition. Whenever a consumer needs to consume a token, there is a producer that has produced one for it. The only situation that requires regeneration is when a producer is outside a loop and its consumer is inside: the producer fires once, but the consumer requires a token on every iteration. This is not a case of a missing producer, but of a single token that must be available multiple times. The regeneration algorithm resolves this by reproducing the token each time the loop iterates.

The algorithm iterates over all producer-consumer deliveries and, for each delivery where the producer is outside a loop that encloses the consumer, inserts a MUX at the header of that loop. The two data inputs of the MUX represent the initial value from outside the loop and

the regenerated value from within:

- The *external input* carries the initial token from outside the loop. On the first iteration, this input is selected.
- The *internal input* carries the token fed back from the MUX output, providing the regenerated copy for subsequent iterations.
- The *select input* is the loop exit condition, combined with an INIT component as described in Section 4.1.

The MUX output is connected to both the consumer inside the loop body and the internal input of the MUX itself. If the consumer is inside multiple nested loops, one MUX is inserted at the header of each enclosing loop, forming a chain: the output of the outer MUX feeds the external input of the next inner MUX.

Figure 4.3 illustrates the mechanism. BB0 defines x and transfers control to BB1, the loop header. BB1 transfers control to BB2, which consumes x and branches back to BB1 (when c_2 is true) or exits to BB3 (when c_2 is false). The producer in BB0 fires once, but the consumer in BB2 requires a token on every iteration. The regeneration MUX at BB1 replaces the original cross-loop delivery with three separate deliveries, none of which crosses a loop boundary from outside to inside:

1. **Producer to external input.** The token from BB0 is delivered to the external input of the MUX. The external input is consumed only on the first iteration, so both endpoints are effectively outside the loop.
2. **Feedback.** The MUX output is fed back to its own internal input. Both endpoints are inside the same loop.
3. **MUX output to consumer.** The MUX output is delivered to the consumer in BB2. Both endpoints are inside the same loop.

All three deliveries may still require suppression. For instance, the feedback delivery must be suppressed when the loop exits, because the MUX output produces a token on the final iteration that the internal input will not consume.

The select signal of the regeneration MUX is the loop exit condition, as with the μ -function select signal in Section 4.1, and the corresponding circuit is constructed using the algorithm developed in Chapter 5.

As the example shows, regeneration converts every loop-entering delivery into deliveries that stay within the same loop level. After all deliveries have been processed, no delivery crosses a loop boundary from outside to inside, and no further regeneration is needed. The only remaining delivery problem is that a producer fires but the consumer does not execute, which requires suppression. The next section defines the suppression problem, and Chapter 5 develops the algorithm in detail.

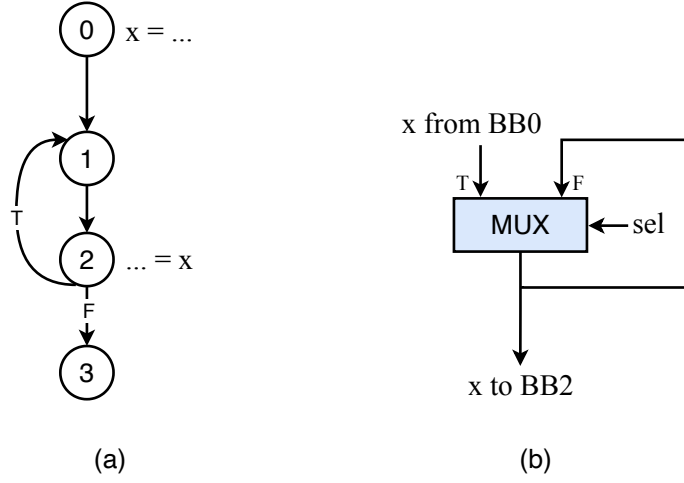


Figure 4.3: Regeneration for a producer outside a loop and a consumer inside. (a) CFG with x defined in BB0 outside the loop and consumed in BB2 inside the loop. (b) A regeneration MUX at the loop header BB1, with external input from BB0, internal feedback, and output to BB2.

4.3 Suppression

Given a producer in block B_p and a consumer in block B_c , the suppression problem is to compute a Boolean condition f_{supp} and construct a circuit that discards the producer's token whenever f_{supp} is true, while satisfying the token-matching invariants for all tokens within the circuit.

The suppression circuit consists of two parts. The first is an *expression circuit* that evaluates f_{supp} from condition tokens produced by the conditional blocks in the CFG. The second is a **BRANCH** that receives the producer's data token and the output of the expression circuit: when f_{supp} is true, the token is forwarded to a **SINK** and discarded, and when false, the token passes through to the consumer.

Equivalently, the expression circuit can compute the consumption condition $f_{\text{cons}} = \overline{f_{\text{supp}}}$, with the **BRANCH** outputs swapped: true passes the token through and false discards it. In the following chapters, we sometimes adopt this formulation when it makes the construction easier to follow.

The expression circuit is the component that Chapter 3 identified as problematic in the existing methodology. The algorithm for constructing it so that it satisfies the token-matching invariants internally is developed in Chapter 5.

Select signals. The select signal of a μ -function and the select signal of a regeneration MUX both represent the loop exit condition, which is equivalent to the suppression condition from the loop header to itself through the back edge. These select signals use the same expression circuit construction, without the **BRANCH**: the expression circuit output drives the MUX select input directly.

4.4 Condition Variable Placeholders

The suppression algorithm in Chapter 5 assumes that each conditional block is the producer of its own condition token: whenever control flow arrives at a conditional block, the block produces a condition token that is forked and sent to all its consumers. In practice, however, the condition variable may be defined by an operation in a different block, whose execution conditions differ from those of the conditional block. To establish the assumed semantics, we treat the conditional branch at the end of the block as a consumer, receiving the condition value from its actual producer. This is an ordinary producer-consumer delivery, and by applying regeneration and suppression to it, the output token arrives exactly when the conditional block is reached, matching the assumption.

However, this leads to an ordering problem. Condition tokens are used throughout the circuit as select signals of γ -trees, μ -functions, regeneration MUX components, and suppression circuits. If a condition variable's own delivery has not yet been through regeneration and suppression, any circuit that reads the condition token directly from its producer will receive a token with the wrong semantics.

A natural idea would be to process all condition deliveries first, completing their regeneration and suppression before building any other circuit. But this is not always possible: the regeneration and suppression circuit for one condition variable may itself require another condition variable as input, and that second condition's regeneration and suppression circuit may in turn require the first. Such circular dependencies make it impossible to find a processing order in which every condition is fully resolved before it is needed.

To decouple the delivery of the condition value to its conditional block from the use of the condition, the algorithm inserts a *placeholder* for each condition variable before any deliveries are processed. A placeholder is a single-input, single-output pass-through component. Its output is connected to all components that use the condition, so that the correct token semantics are in place from the start. Its input is the consumer end of this delivery: when the algorithm processes the condition variable delivery in the regeneration and suppression phases, the result is connected to the placeholder input, just as any other delivery connects to its consumer. After all deliveries have been processed, every placeholder becomes a direct connection, and the circuit is complete.

4.5 Algorithm Overview

Algorithm 1 summarizes the complete FTD procedure described in this chapter.

Phase 1 inserts placeholders so that condition tokens can be referenced by all subsequent phases. Phase 2 converts ϕ -functions to μ -functions and γ -trees, and Phase 3 translates BB operations into elastic components. Phase 4 performs regeneration on all loop-entering deliveries. After Phase 4, no delivery crosses a loop boundary from outside to inside. Phase 5 constructs a suppression circuit for each remaining delivery where the producer fires but the consumer does not execute. Condition variable deliveries are processed in Phase 4 and 5 alongside all other deliveries, with each placeholder serving as the consumer of its condition's delivery. After Phase 5, every delivery in the circuit satisfies the token-matching invariants.

Algorithm 1: Fast Token Delivery

Input: CFG in SSA form**Output:** Elastic dataflow circuit

```

/* Phase 1: Condition Placeholders (Section 4.4) */
1 foreach condition variable  $c_i$  in the CFG do
2   | Insert a placeholder for  $c_i$ 

/* Phase 2: GSA Conversion (Section 4.1) */
3 foreach  $\phi$ -function in the CFG do
4   | if  $\phi$  satisfies the  $\mu$  conditions then
5   |   | Replace  $\phi$  with a  $\mu$ -function
6   |   | Construct circuit for  $\mu$  select signal (Chapter 5)
7   | else
8   |   | Replace  $\phi$  with a  $\gamma$ -tree

/* Phase 3: BB Translation */
9 Translate BB operations into elastic components
10 Connect data-dependent components within each BB

/* Phase 4: Regeneration (Section 4.2) */
11 foreach producer-consumer delivery do
12   | foreach loop enclosing the consumer but not the producer do
13   |   | Insert a regeneration MUX at the loop header
14   |   | Construct circuit for select signal (Chapter 5)

/* Phase 5: Suppression (Section 4.3) */
15 foreach producer-consumer delivery do
16   | Construct suppression circuit (Chapter 5)

```

The Suppression Algorithm

This chapter presents the suppression algorithm introduced in Section 4.3. Given a producer block B_p and a consumer block B_c in the CFG, the algorithm constructs an elastic circuit that discards the producer's token whenever the consumer will not execute, while satisfying the token-matching invariants for all tokens within the circuit.

We first develop the algorithm for the case where the consumer consumes a token whenever its block is reached (Sections 5.1–5.6). Section 5.7 generalizes to consumers on γ -trees, where the consumer block may be reached without the token being consumed.

5.1 Extracting the Producer-Consumer Relationship

In the case where the consumer consumes a token whenever its block is reached, the GSA representation guarantees that the consumer has a unique reaching definition, and that this definition is available on every path leading to the consumer's block. This means the producer's block B_p dominates the consumer's block B_c : every path from the program entry to B_c passes through B_p .

Because of this dominance, the suppression question reduces to a local one. When B_p fires, the portion of the CFG before B_p has already been traversed and is no longer relevant. The only question is whether the control flow, starting from B_p , will reach B_c before returning to B_p or exiting the program. If it does, the token should be delivered. If it does not, the token should be discarded. In particular, if the control flow returns to B_p without passing through B_c , B_p will fire again and produce a new token, and the current token will be overwritten.

The same reasoning extends to two other types of delivery. As described in Sections 4.1 and 4.2, the μ -function and the regeneration MUX each have an internal input that carries a value from inside the loop. The producer of this value is inside the loop body, while the consumer is at the loop header. The producer's block does not dominate the loop header. However, the internal input is consumed only when the loop iterates, not on the first entry. On any iteration where the internal input needs a token, the control flow has traversed the loop body and passed through the producer's block, so the producer has already fired and produced a token. This gives the same guarantee as dominance: whenever the consumer needs a token, the producer has already produced one, and the suppression analysis can focus on the paths from the producer forward.

This motivates the construction of a *local CFG*: a directed graph derived from the original

CFG for the pair (B_p, B_c) . The local CFG captures exactly the paths from the producer to the consumer and redirects all other paths to a single *sink* block representing suppression.

From such a graph, the suppression condition can be expressed as a Boolean function of the branch conditions along the paths. However, the original CFG may contain loops, and a cyclic graph does not directly yield a finite Boolean expression. Before describing the construction of the local CFG, we must understand how each type of loop interacts with the delivery.

Loop Classification

We restrict attention to *reducible* control flow graphs [16], in which every cycle constitutes a natural loop defined by a back edge $n \rightarrow h$ where the header h dominates the tail n . Irreducible loops, where control flow can enter a cycle at multiple nodes, lack a unique loop header and prevent the formation of a canonical loop hierarchy [17, 18]. This restriction is standard in HLS and is satisfied by all CFGs produced by LLVM from structured C programs. Under this assumption, every loop in the CFG has a unique header and the loops form a well-defined nesting hierarchy.

Let L be a natural loop in the original CFG. We classify loops based on the relationship between L , the producer block B_p , and the consumer block B_c .

Case 1: B_p and B_c in the same loop. The treatment depends on the relative order of the two blocks within the loop.

In the *forward* case (Figure 5.1a), B_p precedes B_c within the loop. A valid delivery follows a forward path from B_p to B_c without traversing the back edge. If the control flow takes the back edge instead, it returns to the loop header. From the header, any path to B_c must pass through B_p , because B_p dominates B_c . But passing through B_p means B_p fires again and the current token should be discarded. Therefore, traversing the back edge always indicates suppression.

In the *backward* case (Figure 5.1b), B_c precedes B_p in the loop body, or $B_p = B_c$ with the producer after the consumer. The token can reach B_c only by traversing the back edge and returning to B_c . This case arises only when B_c is the loop header, and corresponds to deliveries to the internal input of a μ -function or a regeneration MUX. To see why B_c must be the loop header, suppose B_c were an interior node of the loop. Then B_c could be reached both from B_p inside the loop and from outside the loop through the header. An outside definition must exist, because otherwise the control flow could enter the loop and reach B_c without passing through B_p , leaving B_c without a reaching definition and violating the GSA property. With two definitions reaching B_c , the GSA conversion would place a μ -function at the loop header to merge them, and the delivery from B_p would go to the μ 's internal input at the header rather than to B_c directly, contradicting the assumption that B_c is the consumer block.

Case 2: B_p in the loop, B_c outside. For the token to be delivered, the control flow must exit the loop and reach B_c (Figure 5.1c). If the control flow takes the back edge instead, B_p

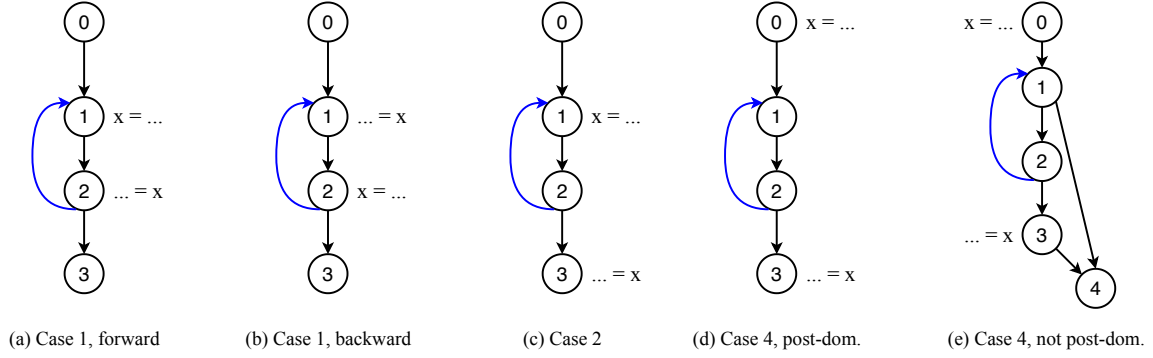


Figure 5.1: Classification of loops.

will fire again on the next iteration and produce a new token. The current token should be discarded. As in Case 1 forward, traversing the back edge indicates suppression.

Case 3: B_c in the loop, B_p outside. This is the regeneration scenario handled in Section 4.2 and does not arise during suppression analysis.

Case 4: Neither B_p nor B_c in the loop. The loop lies on an intermediate path between the producer and the consumer. If B_c post-dominates the loop header (Figure 5.1d), the loop does not affect whether the control flow reaches B_c , and the branch conditions within the loop play no role in the suppression condition. If B_c does not post-dominate the loop header (Figure 5.1e), some branch inside the loop determines whether B_c is reached. The loop may execute multiple iterations, producing a condition token on each iteration, of which only the last one is relevant to the suppression decision.

Handling loops in the local CFG. In the forward subcase of Case 1 and in Case 2, the back edge always indicates suppression. The local CFG construction handles these cases by redirecting back edges to the sink block, making the graph acyclic. For the backward subcase of Case 1, a proxy block for B_c is placed at the target of the back edge, replacing the back edge with a forward edge to the proxy. Since reaching the proxy means the token has been delivered and there is no need to continue through the loop, the proxy is connected to the sink and the loop is eliminated. In Case 4, if B_c post-dominates the loop header, the loop has no effect on the suppression condition and will be optimized away in Section 5.2. If B_c does not post-dominate the loop header, the loop cannot be removed by redirecting the back edge, because doing so would change the semantics: in Cases 1 and 2, traversing the back edge means the current token should be discarded, but in Case 4, traversing the back edge simply means the loop has not yet exited, and the suppression decision is still pending. These loops are preserved in the local CFG and addressed in Section 5.3.

Construction of the Local CFG

The local CFG G' for a pair (B_p, B_c) is constructed by a depth-first traversal of the original CFG starting from B_p . The traversal creates a copy of each visited block and applies the loop transformations described above. A dedicated sink block B_{sink} collects all paths that fail to deliver the token.

Algorithm 2 describes the construction. For each successor S of the current block, the algorithm evaluates the following cases in priority order:

1. **Consumer reached.** If $S = B_c$, a proxy block for B_c is created and connected to B_{sink} . This proxy becomes the local consumer B'_c . Since the delivery outcome is determined at this point, no further traversal beyond B'_c is needed.
2. **Producer re-entry.** If $S = B_p$, the control flow has returned to the producer without reaching the consumer. The current block is connected to B_{sink} .
3. **Already-visited block.** If S has been visited and is neither B_p nor B_c , the edge is added to G' and the algorithm simply backtracks without expanding S again. This preserves Case 4 loops.
4. **Back edge.** If the edge is a back edge to an unvisited block, the token is suppressed, and the current block is connected to B_{sink} .
5. **Normal forward edge.** A copy of S is created, the edge is added, and the traversal continues recursively.

If a block has no successors (the program exit), it is connected to B_{sink} .

Figure 5.2 illustrates the construction of the local CFG (Figure 5.2b) from an original CFG (Figure 5.2a) with $B_p = \text{BB2}$ and $B_c = \text{BB1}$. This example involves all three cases that arise during suppression analysis. The CFG contains three loops: an innermost loop between BB5 and BB6, which lies on an intermediate path between B_p and B_c (Case 4), a middle loop from BB2 to BB7, where B_p is inside the loop and B_c is outside (Case 2), and an outermost loop from BB1 to BB8, where both B_p and B_c are in the loop with B_p after B_c (Case 1, backward).

The DFS starts at BB2 and creates B'_p . Its successors BB3 and BB4 are unvisited forward edges, so copies are created for both. BB3 and BB4 each lead to BB5, which leads to BB6. BB6 has two successors: its true branch goes to BB7, and its false branch returns to BB5. BB5 has already been visited and is neither B_p nor B_c , so the edge is added without revisiting BB5. This preserves the innermost loop as a Case 4 loop. From BB7, the true branch goes to BB8, and the false branch returns to BB2. Since BB2 is B_p , this edge is redirected to B_{sink} . From BB8, the true branch goes to BB1, which is B_c . A proxy block B'_c is created and connected to B_{sink} . The false branch goes to BB9, which has no successors and is connected to B_{sink} .

The CDG and decision graph derived from this local CFG (Figure 5.2c, d) are discussed in Section 5.2.

Algorithm 2: Build Local CFG

Input: Original CFG G , producer block B_p , consumer block B_c **Output:** Local CFG G' with local producer B'_p and local consumer B'_c

```

1 Add sink block  $B_{\text{sink}}$  to  $G'$ ;
2 Mark all blocks in  $G$  as unvisited;
3 Function DFS( $B$ ):
4   mark  $B$  as visited;
5   foreach successor  $S$  of  $B$  do
6     if  $S = B_c$  then
7       create block  $S'$  in  $G'$ ;
8       add edge  $(B' \rightarrow S')$  and  $(S' \rightarrow B_{\text{sink}})$ ;
9       set  $B'_c \leftarrow S'$ ;
10    else if  $S = B_p$  then
11      add edge  $(B' \rightarrow B_{\text{sink}})$ ;
12    else if  $S$  is visited then
13      add edge  $(B' \rightarrow S')$ ;
14    else if edge  $(B \rightarrow S)$  is a back edge then
15      add edge  $(B' \rightarrow B_{\text{sink}})$ ;
16    else
17      create block  $S'$  in  $G'$ ;
18      add edge  $(B' \rightarrow S')$ ;
19      DFS( $S$ );
20  if  $B$  has no successors then
21    add edge  $(B' \rightarrow B_{\text{sink}})$ ;
22 create block  $B'_p$  in  $G'$  corresponding to  $B_p$ ;
23 DFS( $B_p$ );

```

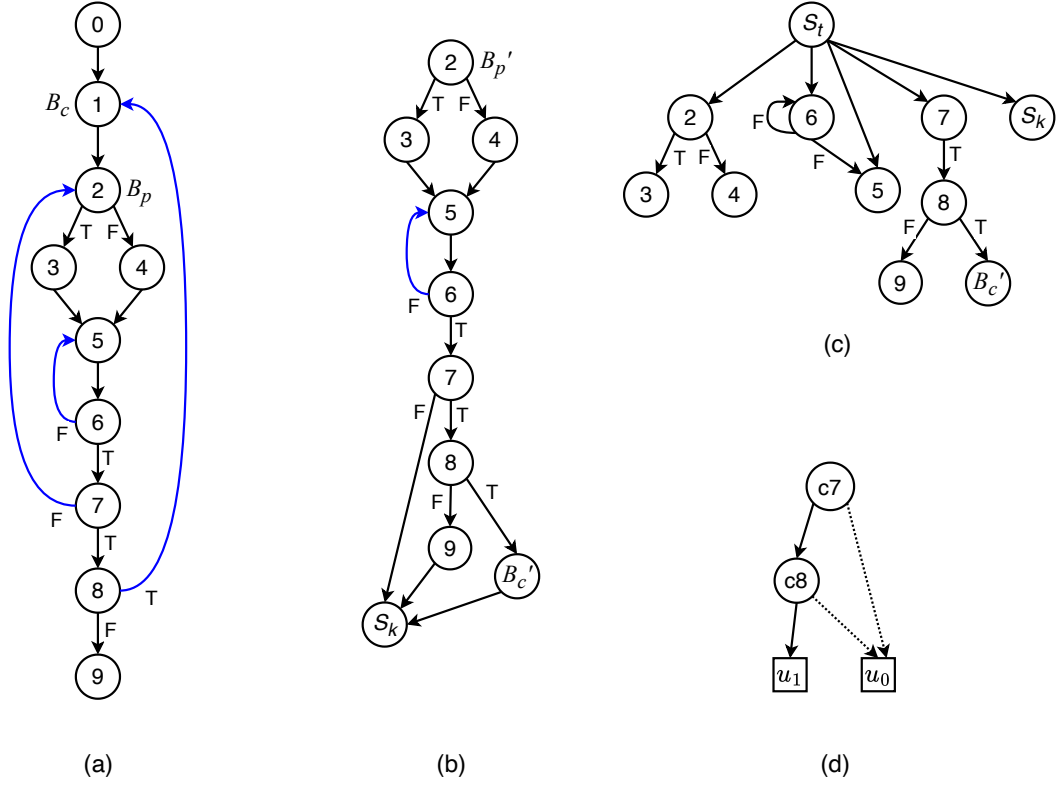


Figure 5.2: Construction of the local CFG, its CDG, and the decision graph from the original CFG. (a) The original CFG. Blue edges are back edges. (b) The local CFG. S_k denotes B_{sink} . (c) The CDG of the local CFG. S_t denotes the entry node and S_k denotes B_{sink} . (d) The decision graph. Dashed edges denote false edges.

Properties of the Local CFG

The local CFG G' has the following properties:

1. G' has a unique source (B_p') and a unique sink (B_{sink}). Every path from B_p' reaches B_{sink} .
2. Apart from Case 4 loops, G' is a directed acyclic graph.
3. A path from B_p' to B_{sink} that passes through B_c' represents a successful delivery. A path that reaches B_{sink} without passing through B_c' represents suppression.
4. The suppression condition f_{supp} is true if and only if the control flow follows a path that reaches B_{sink} without passing through B_c' .

5.2 The Decision Graph

The local CFG encodes the suppression condition in the structure of its paths, but not all of its blocks contribute to this condition. A block with a single successor imposes no condition on the control flow. A block with two successors where both branches lead to the same outcome does not affect whether the consumer is reached. The only blocks whose branch conditions appear in the suppression expression are those in the transitive closure of the control dependence of B'_c in the local CFG.

This section extracts these blocks into a compact graph called the *decision graph*, which retains only the information relevant to the suppression condition and discards everything else. Let

$$\mathcal{C} = \{ B \in V' \mid B'_c \text{ is transitively control dependent on } B \}$$

denote the set of blocks in the local CFG whose branch conditions appear in the suppression expression. Each block $B \in \mathcal{C}$ is associated with the Boolean variable c_B of its conditional branch. Before constructing the decision graph, we establish three structural properties of the blocks in $\mathcal{C}$ that follow from the definition of control dependence.

Property 1: Binary successors. Every block $B \in \mathcal{C}$ has exactly two successors.

Proof. A block with zero or a single successor makes no branch decision and cannot affect which blocks are reached, so the control dependence condition is not satisfied. $\square$

Property 2: Successor post-dominance. For each successor S of a block $B \in \mathcal{C}$, either S is post-dominated by some block in $\mathcal{C} \cup \{B'_c\}$, or S cannot reach any block in $\mathcal{C} \cup \{B'_c\}$.

Proof. Suppose S can reach at least one block in $\mathcal{C} \cup \{B'_c\}$, and let T be the first such block along a path from S . If T did not post-dominate S , there would exist a path from S that avoids T . Along this path, T is control dependent on some block that should also belong to $\mathcal{C}$, contradicting that T is the first block in $\mathcal{C} \cup \{B'_c\}$ along the path from S . Therefore T post-dominates S . $\square$

Property 3: Successor divergence. For any block $B \in \mathcal{C}$, its two successors cannot both be post-dominated by the same block $T \in \mathcal{C} \cup \{B'_c\}$.

Proof. If both successors of B were post-dominated by T , then T would post-dominate B itself. This contradicts the control dependence property. $\square$

Property 4: Direct post-dominance. For a successor S of a block $B \in \mathcal{C}$ that is post-dominated by some $T \in \mathcal{C} \cup \{B'_c\}$, no other block in $\mathcal{C} \cup \{B'_c\}$ lies between S and T .

Proof. Suppose a block $U \in \mathcal{C}$ is reachable from S before reaching T . Since T post-dominates S , it also post-dominates U . Therefore, $U \notin \mathcal{C}$. $\square$

Property 5: Reachability. For any block $B \in \mathcal{C}$, at least one successor of B can reach a block in $\mathcal{C} \cup \{B'_c\}$.

Proof. Control dependence requires a path from the controlling block to the dependent block. Transitive control dependence preserves this reachability. $\square$

Construction of the Decision Graph

The decision graph $D = (U, A, r)$ is constructed as follows (Algorithm 3). The vertex set U contains one nonterminal vertex for each block in $\mathcal{C}$, plus two terminal vertices: u_1 (representing B'_c , successful delivery) and u_0 (representing B_{sink} , suppression). The edge set A is induced by the children of each nonterminal vertex, as described below. The root r is determined by the traversal starting from B'_p .

For each nonterminal vertex corresponding to a block $B \in \mathcal{C}$, the **high** child (when c_B is true) and the **low** child (when c_B is false) are determined by walking forward from each successor of B in the local CFG. By Properties 2 and 4, this walk either reaches exactly one block in $\mathcal{C} \cup \{B'_c\}$ that post-dominates the successor, or reaches B_{sink} without encountering any such block. In either case, which particular path the walk follows does not matter, because the post-dominance guarantees the same outcome. If the walk reaches a block in $\mathcal{C}$ that has not yet been visited, a new nonterminal vertex is created and the construction recurses on its two successors. If the block has already been visited, an edge is added to the existing vertex. If the walk reaches B'_c , the child is set to u_1 . If the walk reaches B_{sink} , the child is set to u_0 .

Note that B'_p may or may not belong to $\mathcal{C}$. If $B'_p \in \mathcal{C}$, it becomes the root. Otherwise, the walk from B'_p reaches the first block in $\mathcal{C}$, which becomes the root.

Figure 5.2c shows the CDG of the local CFG from the running example. Although the local CFG contains many blocks, only BB7 and BB8 appear in the transitive closure of the control dependence of B'_c . Therefore $\mathcal{C} = \{\text{BB7}, \text{BB8}\}$.

The algorithm starts at $B'_p = \text{BB2}$, which is not in $\mathcal{C}$. Walking forward, it first reaches BB7, which is in $\mathcal{C}$. BB7 becomes the root of the decision graph. The true successor of BB7 leads to BB8, also in $\mathcal{C}$, so a second nonterminal vertex is created. The false successor of BB7 leads to B_{sink} , connecting to u_0 . From BB8, the true successor leads to B'_c , connecting to u_1 , and the false successor leads to B_{sink} , connecting to u_0 .

The resulting decision graph (Figure 5.2d) has two nonterminal vertices (c_7 and c_8) and two terminal vertices.

ROBDD Properties

If the decision graph is acyclic, it is an ROBDD.

The *ordered property* follows from the topological order of the original CFG. Each nonterminal vertex corresponds to a block in $\mathcal{C}$, and these blocks inherit their indices from the original CFG. Along any path from root to terminal, the indices strictly increase.

The *reduced property* requires two conditions. First, no nonterminal vertex has $\text{low}(u) = \text{high}(u)$, which follows from Property 3 and Property 5. Second, no two distinct vertices root

Algorithm 3: Build Decision Graph from Local CFG

Input: Local CFG G' , control dependence set $\mathcal{C}$, local consumer B'_c , sink B_{sink}

Output: Decision graph $D = (U, A, r)$

```

1 Create terminal vertices  $u_0$  and  $u_1$ ;
2 Create an empty map  $\text{vertex} : \mathcal{C} \rightarrow U$ ;
3 Function Walk( $B$ ):
4   if  $B = B'_c$  then
5     return  $u_1$ ;
6   else if  $B = B_{\text{sink}}$  then
7     return  $u_0$ ;
8   else if  $B \in \mathcal{C}$  then
9     if  $\text{vertex}(B)$  exists then
10      return  $\text{vertex}(B)$ ;
11     create nonterminal vertex  $u_B$  with variable  $c_B$ ;
12      $\text{vertex}(B) \leftarrow u_B$ ;
13     let  $S_T, S_F$  be the true and false successors of  $B$ ;
14      $\text{high}(u_B) \leftarrow \text{Walk}(S_T)$ ;
15      $\text{low}(u_B) \leftarrow \text{Walk}(S_F)$ ;
16     return  $u_B$ ;
17   else
18     //  $B \notin \mathcal{C}$ : walk forward to the next relevant block
19     let  $S$  be a random successor of  $B$ ;
20     return Walk( $S$ );
21  $r \leftarrow \text{Walk}(B'_p)$ ;

```

isomorphic subgraphs, because each vertex corresponds to a distinct block with a distinct index, and the subgraph rooted at each is uniquely determined.

The decision graph therefore satisfies both properties O and R and is an ROBDD. By the canonicity theorem [14], it is the unique reduced representation of the suppression function under this variable ordering.

In the running example, the decision graph is acyclic, which is therefore an ROBDD. The suppression condition $f_{\text{supp}} = \overline{c_7} + c_7 \cdot \overline{c_8} = \overline{c_7 \cdot c_8}$ can be implemented as a MUX-tree using the Shannon expansion mapping from Section 2.4.

When the decision graph contains cycles, it is not an ROBDD and cannot be directly mapped to a circuit. This happens when the local CFG has a Case 4 loop whose header is not post-dominated by B'_c , so that a block inside the loop belongs to $\mathcal{C}$ and the corresponding back edge appears in the decision graph. The next section addresses this case.

5.3 Decomposing Cyclic Decision Graphs

When the decision graph contains cycles, it cannot be directly mapped to a MUX-tree. The cycles mean that condition tokens may be produced over multiple iterations of a loop, but the suppression circuit needs only the token from the final iteration when the loop exits. All earlier tokens have no consumer and must be filtered out to satisfy the token-matching invariants.

We illustrate the idea on a simple case before developing the general algorithm. Figure 5.3a shows the CFG from Figure 5.1e with $B_p = \text{BB0}$ and $B_c = \text{BB3}$. BB1 and BB2 form a loop on the path from the producer to the consumer. The decision graph (Figure 5.3b) has two nonterminal vertices, c_1 and c_2 , with a back edge from c_2 to c_1 .

If we remove the back edge, the graph becomes acyclic and represents the behavior of the final iteration, when the loop exits. On this iteration, if c_1 is true, c_2 must be true (otherwise the loop would not have exited), and the control flow reaches B_c and the token should be delivered. If c_1 is false, the control flow reaches B_{sink} . The acyclic suppression condition is therefore $\overline{c_1}$.

Figure 5.3c shows the suppression circuit. The main **BRANCH** receives the producer's token on its data input and c_1 on its select input. When c_1 is true, the token passes through to the consumer. When c_1 is false, the token is discarded. However, BB1 fires on every iteration of the loop, producing a c_1 token each time, while the producer fires only once. Connecting c_1 directly to the main **BRANCH** would leave all but one of these tokens unconsumed.

The solution is to filter the c_1 tokens before they reach the main **BRANCH**. A second **BRANCH** receives the c_1 token on its data input and the loop exit condition $(\overline{c_1} + c_1 c_2)$ on its select input. When the exit condition is false, the loop is still iterating and the c_1 token is discarded. When the exit condition is true, the loop has exited and the c_1 token is passed through to the main **BRANCH**.

This filtering does not introduce a new token mismatch. The loop exit condition is evaluated on every iteration, so a token for its select input is always consumed whenever it arrives. After filtering, exactly one c_1 token reaches the main **BRANCH**, matching the single token from the producer.

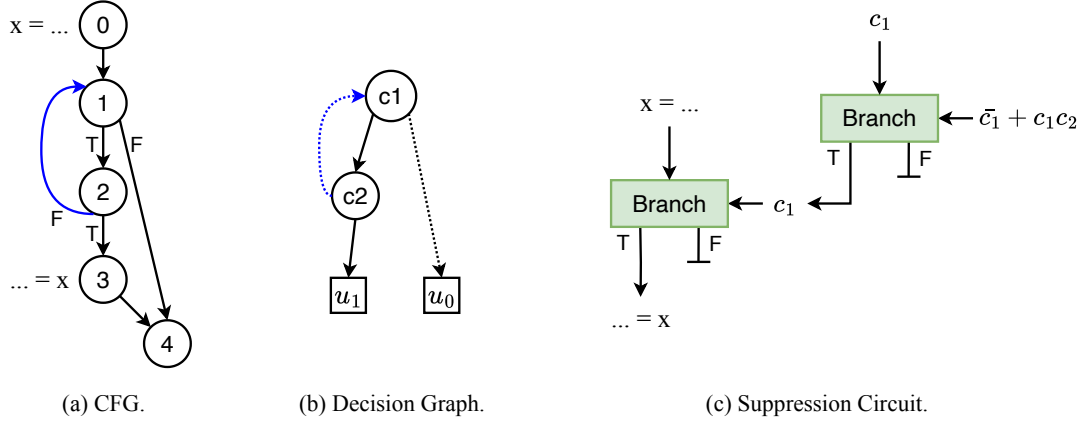


Figure 5.3: A simple cyclic decision graph and its suppression circuit for $B_p = \text{BB0}$ and $B_c = \text{BB3}$. (a) The CFG with a loop between BB1 and BB2. (b) The decision graph. The back edge from c_2 to c_1 creates a cycle. Dashed edges denote false edges. (c) The suppression circuit. The right **BRANCH** filters c_1 tokens using the loop exit condition. The left **BRANCH** evaluates the acyclic suppression condition.

The simple example above established the approach: remove the back edge to obtain an acyclic expression for the final iteration, and filter condition tokens using the loop exit condition so that only the last iteration's tokens reach the acyclic circuit. We now develop this idea into a systematic algorithm that handles arbitrary nesting of loops in the decision graph.

Nesting Level

For a reducible decision graph, the nesting structure is defined by its back edges. Each back edge $n \rightarrow h$ targets a loop header h . The natural loop of h consists of h and all vertices that can reach the tail of some back edge targeting h without passing through h . Multiple back edges may target the same header and together define a single loop. The *nesting level* of a vertex is the number of loops that contain it. Reducibility guarantees that loops at the same level are disjoint and that no inner loop is shared between two distinct outer loops.

Figure 5.4 shows a more complex example with $B_p = \text{BB0}$ and $B_c = \text{BB8}$ that will serve as the running example through Sections 5.3–5.6. The decision graph (Figure 5.4b) has seven nonterminal vertices c_1 through c_7 and contains two loops. The outer loop has header c_1 and back edge from c_4 to c_1 , and contains c_1, c_2, c_3, c_4, c_5 . We call this level 1. Inside it, the inner loop has header c_3 and back edge from c_5 to c_3 , and contains c_3, c_5 . We call this level 2. Level 0 refers to the entire decision graph. The nesting level of a vertex is the deepest loop that contains it: c_6 and c_7 are at level 0, c_1, c_2 , and c_4 are at level 1, and c_3 and c_5 are at level 2. A vertex at level k lies within the loops at levels $k, k-1, \dots, 1$ and within the level-0 region.

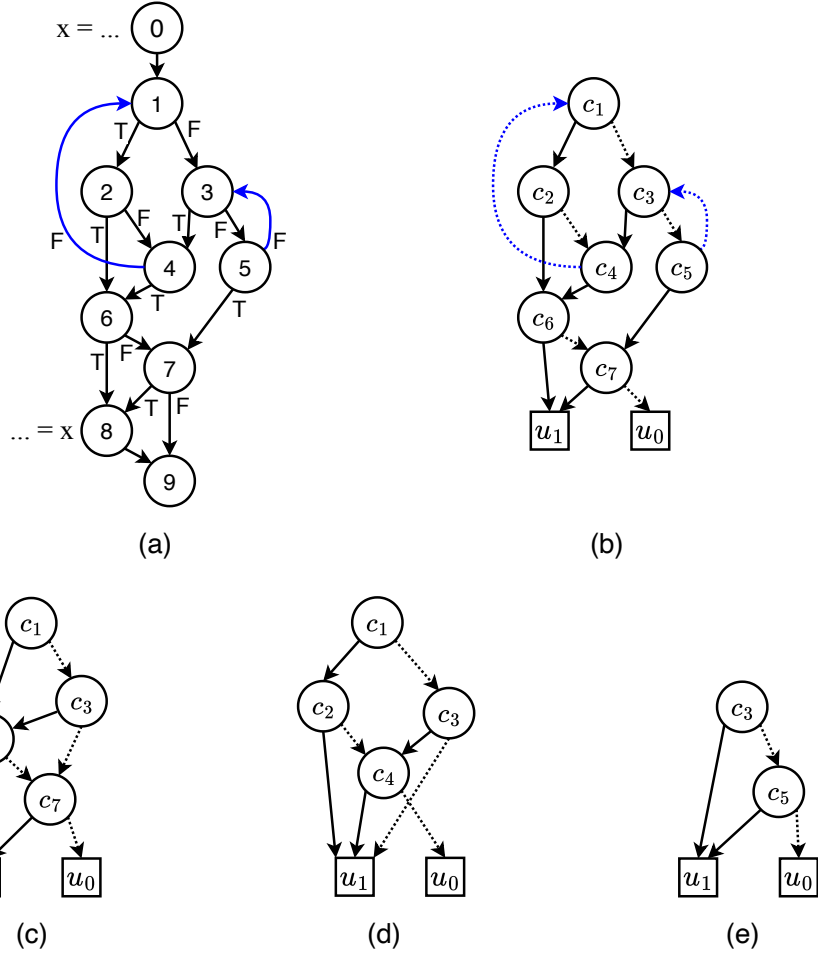


Figure 5.4: Decomposition of a cyclic decision graph for $B_p = \text{BB0}$ and $B_c = \text{BB8}$. Dashed edges denote false edges. (a) The CFG with two nested loops. (b) The cyclic decision graph. (c) Level-0 ROBDD. (d) Level-1 ROBDD. (e) Level-2 ROBDD.

Level ROBDD Construction

For each loop at nesting level $k \geq 1$, we construct an ROBDD that determines, on each iteration, whether the control flow exits the loop or returns to the header.

The level- k ROBDD for a loop with header h is constructed from the vertices within its region as follows.

- Edges to vertices outside the loop or to terminals are redirected to the true terminal, representing a loop exit.
- The back edge to the header is redirected to the false terminal, representing another iteration.

- Back edges belonging to higher nesting levels are omitted. Each level- k ROBDD assumes that the tokens it receives from higher-level vertices have already been filtered by the corresponding higher-level ROBDDs. Under this assumption, the level- k ROBDD operates as if the graph were acyclic, which is necessary for the ROBDD-to-circuit mapping to produce a correct result.
- After the above steps, the graph is acyclic. Vertices with fewer than two remaining successors or whose two successors point to the same target are recursively eliminated to make the graph reduced.

After these steps, the result is an ROBDD.

The level-0 ROBDD is a special case. It is obtained by removing all back edges from the decision graph and reducing the result. Since the terminal vertices u_0 and u_1 already represent suppression and delivery, no edges need to be redirected. The level-0 ROBDD is the final suppression expression: it evaluates f_{supp} using condition tokens that have been filtered through the higher-level ROBDDs, ensuring that each token corresponds to the last iteration of its loop.

We now illustrate how the decision graph of Figure 5.4b is decomposed into the three level ROBDDs shown in Figure 5.4c, d, and e.

The *level-0 ROBDD* (Figure 5.4c) is obtained by removing all back edges from the decision graph. Without the back edge from c_4 to c_1 , both c_1 (true) and c_3 (true) lead unconditionally to c_6 , so c_2 and c_4 are redundant and eliminated. Similarly, without the back edge from c_5 to c_3 , c_3 (false) leads unconditionally to c_7 , so c_5 is redundant and eliminated. The level-0 ROBDD has four nonterminal vertices: c_1, c_3, c_6, c_7 .

The *level-1 ROBDD* (Figure 5.4d) is constructed from the outer loop with header c_1 . The inner loop's back edge from c_5 to c_3 is omitted. The three exit edges (c_2 true, c_4 true, c_5 true) are redirected to the true terminal. The back edge from c_4 (false) to c_1 is redirected to the false terminal. With only one remaining edge, c_5 is eliminated. The level-1 ROBDD has four nonterminal vertices: c_1, c_2, c_3, c_4 .

The *level-2 ROBDD* (Figure 5.4e) is constructed from the inner loop with header c_3 . There are no higher-level back edges to omit. The two exit edges (c_3 true, c_5 true) are redirected to the true terminal. The back edge from c_5 (false) to c_3 is redirected to the false terminal. The graph is already reduced. The level-2 ROBDD has two nonterminal vertices: c_3, c_5 .

Algorithm 4 summarizes the construction.

Relationship Between Levels

The level ROBDDs define a chain of filtering stages through which each condition token must pass. A condition token produced by a vertex at nesting level k can be used directly as an input to the level- k ROBDD. After being filtered by the level- k ROBDD, the token becomes valid at level $k-1$ and can serve as an input to the level- $(k-1)$ ROBDD, provided that the variable appears in it. This process continues level by level down to level 0.

If a vertex belongs to level k and does not appear in the ROBDD at some level $j \leq k$, then it does not appear in the ROBDD at any level below j either. This is because a lower-level

ROBDD removes more back edges than a higher one, which can only cause more vertices to become redundant during reduction. Conversely, if a variable is needed at some level, it is guaranteed to appear in all ROBDDs between that level and the variable's own nesting level, so the token can always be obtained through successive demotion.

This section has established the ROBDD-level structure of the decomposition. Starting from the next section, we address how to realize each level ROBDD as an elastic circuit and how to implement the demotion between levels, ensuring that the token-matching invariants are satisfied throughout.

Algorithm 4: Construct Level ROBDDs

Input: Cyclic decision graph D , nesting level information

Output: One ROBDD for each loop

```

1 foreach nesting level  $k$ , from highest to lowest do
2   foreach loop  $L$  at level  $k$  with header  $h$  do
3     take  $h$  as the root of the level- $k$  ROBDD for  $L$ ;
4     foreach vertex  $v$  in  $L$  reachable from  $h$  do
5       foreach child  $w$  of  $v$  do
6         if the edge to  $w$  is a back edge of a higher level then
7           omit this edge;
8         else if  $w$  is outside  $L$  or  $w$  is a terminal then
9           redirect this edge to the true terminal;
10        else if  $w = h$  then
11          redirect this edge to the false terminal;
12      reduce the resulting graph to obtain an ROBDD;

```

5.4 From ROBDD to Circuit

The previous section decomposed the decision graph into a level-0 ROBDD and one ROBDD per loop. Each of these ROBDDs is mapped to a MUX-tree using the Shannon expansion of Section 2.4. The level-0 MUX-tree is the expression circuit that drives the select input of the suppression **BRANCH** (equivalently, after swapping the **BRANCH** outputs, it computes f_{cons} , as noted in Section 4.3). The higher-level MUX-trees realize the per-loop exit conditions used for the cross-level filtering.

This mapping does not on its own produce a token-matched MUX-tree. Chapter 3 showed that a MUX-tree constructed from a Boolean expression may evaluate the correct function while still leaving condition tokens unconsumed at some of its inputs. We must therefore determine when the MUX-tree obtained directly from an ROBDD satisfies the token-matching invariants internally, and how to repair it when it does not. We begin with the level-0 MUX-tree.

The Read-Once Property

A Boolean formula is *read-once* if each input variable appears on at most one leaf of the formula tree [19]. In circuit terms, every variable signal has fan-out one and feeds a single gate input. Bshouty et al. [20] extended this notion to generalized bases, where any Boolean function of bounded fan-in may serve as a primitive gate. Under a basis that includes the three-input if-then-else operator $\text{ITE}(s, a, b) = s \cdot a + \bar{s} \cdot b$, a MUX is a single gate, and a MUX-tree is read-once when every condition variable labels at most one MUX input.

A read-once level-0 MUX-tree is token-matched internally. Since B_p dominates B_c , it also dominates every block in $\mathcal{C}$: any path from the program entry to such a block can be extended to a path reaching B_c , and every such path must pass through B_p . Consequently, no control dependence block produces a condition token between two consecutive firings of B_p except through a path that starts at B_p itself. The decomposition of Section 5.3 further ensures that the level-0 MUX-tree receives only the tokens that survive the higher-level filtering, so each level-0 variable receives at most one token per firing of B_p . The execution selects a unique root-to-terminal path in the level-0 ROBDD, and the ordered property of the ROBDD ensures that no vertex appears twice on this path. A level-0 variable therefore receives a token if and only if its vertex lies on the active path.

Evaluating the MUX-tree on a given firing of B_p amounts to traversing the active path in the ROBDD: each MUX on the path passes its evaluation along its selected data input, and the final input produces the tree's single output token. Each nonterminal vertex on the active path has its variable read exactly once in this traversal, at the single MUX input labelled with that variable. Because the MUX-tree is read-once, this input is also the only input in the entire tree connected to that variable. The single token produced by the vertex therefore arrives at its one destined input and is consumed there, with no other input in the tree waiting for it. Vertices off the path produce no token, and no MUX input expects one from them. Every condition token is consumed exactly once, and the MUX-tree is internally token-matched. Also, the output token of the MUX-tree pairs with the single data token from B_p at the suppression BRANCH, so no token is left unconsumed on either side.

If the MUX-tree is not read-once, some condition variable c_B labels two or more MUX inputs, and a FORK delivers a copy of the token to each of them. Only one copy is consumed on any given execution, and the rest are never consumed. This is the failure mode identified in Chapter 3.

In our running example, the level-0 ROBDD of Figure 5.5a has four nonterminal vertices. Vertex c_6 is reachable from the root by two distinct paths, $c_1 \rightarrow c_6$ and $c_1 \rightarrow c_3 \rightarrow c_6$, and vertex c_7 by three. The Shannon expansion duplicates these vertices across the corresponding subtrees, so the MUX-tree in Figure 5.5b places c_6 at two MUX inputs and c_7 at three. Connecting the tokens to this MUX-tree directly would leave some of them unconsumed on every execution.

Splitting to Achieve Read-Once Form

To obtain a read-once MUX-tree, we transform the ROBDD so that no nonterminal vertex is shared between the two subtrees produced at any Shannon expansion step. At the root r , Shannon expansion produces the subtrees rooted at $\text{low}(r)$ and $\text{high}(r)$. If a nonterminal

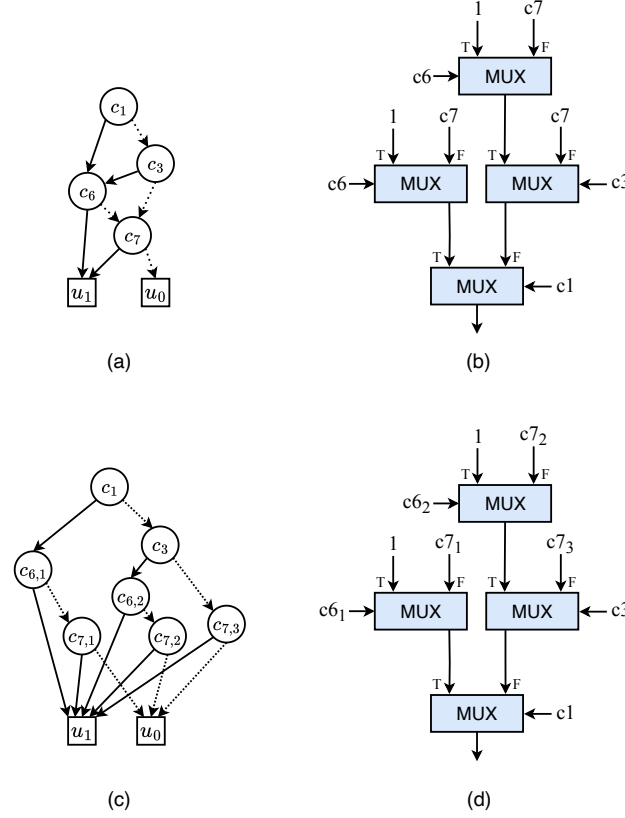


Figure 5.5: Level-0 ROBDD from the running example and its corresponding MUX-tree. (a) The ROBDD has four nonterminal vertices; vertex c_6 is reachable from the root by two distinct paths and c_7 by three. (b) The MUX-tree obtained directly from (a) is not read-once: c_6 appears at two MUX inputs and c_7 at three. (c) The ROBDD after splitting, so that every vertex is reachable by a unique path. (d) The resulting MUX-tree is read-once.

vertex v appears in both, we create two copies of v , one in each subtree, each retaining the children of the original, and treat the copies as labelled with distinct variables for the purpose of the read-once property. The same transformation is applied recursively within each subtree. A vertex that has been split at one step and reappears in both subtrees at a deeper step is split again. Terminal vertices are not duplicated, as they produce no condition token. Algorithm 5 formalizes this procedure. After all splits, the two subtrees at every Shannon expansion step are disjoint in their nonterminal vertices, and since each copy counts as its own variable, the MUX-tree derived from the result is read-once.

Figure 5.5c shows the result for our running example. At the root expansion on c_1 , vertex c_6 appears in both subtrees and is split into $c_{6,1}$ and $c_{6,2}$. Vertex c_7 is split into $c_{7,1}$ on the low side of the root, and further into $c_{7,2}$ and $c_{7,3}$ at the c_3 expansion on the high side. Each split copy is treated as a distinct variable, so in the MUX-tree of Figure 5.5d every variable labels at most one MUX input and the tree is read-once.

Algorithm 5: Split ROBDD to Read-Once Form**Input:** ROBDD $D = (U, A, r)$ **Output:** Split ROBDD in which every nonterminal vertex has a unique root-to-vertex path

```

1 Function Split( $v$ ):
2   if  $v$  is a terminal then
3     return  $v$ ;
4   create a new nonterminal vertex  $v'$  labelled with the same variable as  $v$ ;
5    $\text{low}(v') \leftarrow \text{Split}(\text{low}(v))$ ;
6    $\text{high}(v') \leftarrow \text{Split}(\text{high}(v))$ ;
7   return  $v'$ ;

8  $r' \leftarrow \text{Split}(r)$ ;
9 return the split ROBDD with root  $r'$ ;
```

5.5 Token Distribution

Splitting the ROBDD gives a read-once MUX-tree, but the condition tokens produced by the CFG are not yet ready for it. In the running example, BB6 produces a single c_6 token per firing and BB7 a single c_7 token, whereas the MUX-tree expects tokens at the inputs of its split copies $c_{6,1}, c_{6,2}, c_{7,1}, c_{7,2}, c_{7,3}$. A *distribution circuit* routes each original token to the split input that the execution will consume.

Distribution Branch Tree Construction

For each original variable that has more than one copy in the split ROBDD, we construct a tree of *distribution branches* that routes the original token to the copy visited by the execution.

For a variable c , consider the set of its copies in the split ROBDD. Their least common ancestor (LCA) is the vertex from which all copies are reachable. At the LCA, the copies split into two groups according to which side (low or high) of the LCA leads to each copy. The LCA becomes the root of the distribution branch tree: a **BRANCH** whose select signal is the variable of the LCA, whose data input receives the original c token from the block where c is produced, and whose false and true outputs carry the token toward the two groups. The construction is then applied recursively within each group. If a group also contains more than one copy, the LCA of these copies provides the next **BRANCH** and splits the group further. If a group contains a single copy, the corresponding output is connected to the MUX input associated with that copy.

Figure 5.6 shows the distribution circuits for the running example, whose split ROBDD and MUX-tree are shown in Figure 5.5c and d. The distribution branch trees are highlighted in green. The two copies of c_6 have LCA c_1 , with $c_{6,1}$ reachable from the high side and $c_{6,2}$ from the low side. A single **BRANCH** with select c_1 routes the c_6 token to $c_{6,1}$ on the true output and to $c_{6,2}$ on the false output (Figure 5.6a). The three copies of c_7 have LCA c_1 :

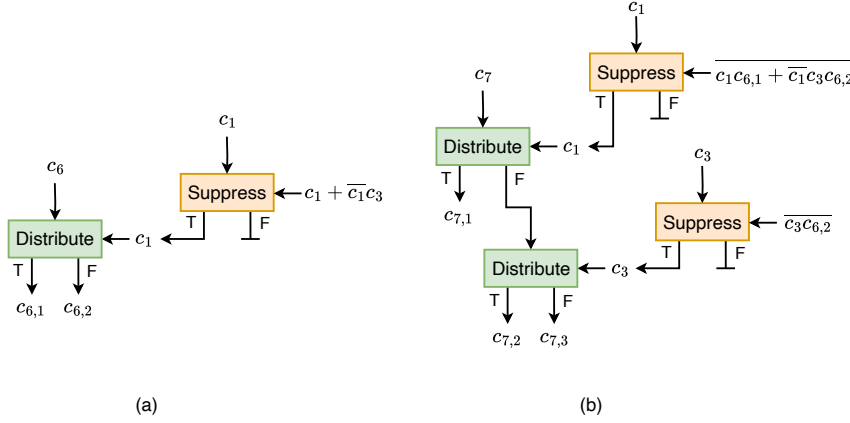


Figure 5.6: Distribution circuits for the running example. Each distribution **BRANCH** has a **SUPPRESS** on its select input. (a) Circuit for c_6 , with one distribution **BRANCH** (select c_1) and a suppression **BRANCH**. (b) Circuit for c_7 , with two distribution **BRANCH**es (select c_1 and c_3), each with a suppression **BRANCH**.

$c_{7,1}$ is reachable from the high side, and $\{c_{7,2}, c_{7,3}\}$ from the low side. The LCA of $c_{7,2}$ and $c_{7,3}$ is c_3 , with $c_{7,2}$ on the high side and $c_{7,3}$ on the low side. The distribution branch tree for c_7 therefore has two **BRANCH**es, with c_1 at the root and c_3 on its false output (Figure 5.6b).

Correctness. On any execution, the split ROBDD is traversed along a unique root-to-terminal path, and exactly one copy of c lies on this path. We verify three properties of the distribution branch tree.

First, the original c token is routed to exactly one leaf. Each **BRANCH** forwards its data token to one of its two outputs according to its select value, so a single input token produces a single leaf output.

Second, this leaf is the one corresponding to the copy on the execution path. Every **BRANCH** in the tree has as its select signal the variable of an LCA vertex, which lies on every path from the root to any copy of c and therefore on the execution path. The select token takes the value of the direction the execution follows at the LCA, which is also the side containing the reached copy. The **BRANCH** forwards the data token toward that side, and applying the same argument recursively, the token arrives at the leaf of the copy on the execution path.

Third, no other copy receives a token, since the **BRANCH** tree is a routing structure with no fan-out on the data path.

Each copy's **MUX** input therefore receives a token on exactly the executions where its vertex is reached, confirming that the copies realize as distinct variables in the sense of Section 5.4. The **MUX**-tree is read-once over these variables, and its internal token-matching follows from the argument of that section. The only remaining concern is the **BRANCH** tree itself: each **BRANCH** receives a select token whenever its LCA vertex is reached, but a data token only when a copy of c is reached below that LCA, and these two conditions do not coincide. The following discussion addresses this mismatch.

Suppressing the Select Inputs

The distribution branch tree routes the data token correctly, but it introduces a new token mismatch at the **BRANCH** components. Each **BRANCH**'s select signal comes from an LCA vertex, and since the LCA lies on every path from the root to the copies below it, the select token is produced whenever the data token is produced. However, the select token may also be produced when no copy of the data variable is reached, and in this case the select token is never consumed. A **SUPPRESS** component is inserted on each select input to discard the select token on exactly these executions.

The suppression expression for a given **BRANCH** is constructed from the sub-ROBDD of the split ROBDD rooted at the **BRANCH**'s select vertex. This sub-ROBDD is also read-once by the projection closure of read-once formulas [20]. From this sub-ROBDD, every edge that leads to a copy of the data variable is redirected to the true terminal, and every edge that leads to a terminal or to a vertex from which no copy is reachable is redirected to the false terminal. This operation only removes vertices and redirects edges to terminals, so the result remains read-once. Its MUX-tree, built by Shannon expansion, satisfies the token-matching invariants without further splitting or distribution. No new mismatch is introduced at any stage of the distribution circuit, and the construction terminates.

The distribution circuits in Figure 5.6 include a **SUPPRESS** on each select input. For c_6 (Figure 5.6a), the single distribution **BRANCH** has select c_1 . Its suppress condition is derived from the sub-ROBDD of the split ROBDD (Figure 5.5c) rooted at c_1 , by redirecting every edge leading to a copy of c_6 to the true terminal. From c_1 's high side, $c_{6,1}$ is reached directly. From c_1 's low side, $c_{6,2}$ is reached through c_3 's high side. The reaching condition is therefore $c_1 + \bar{c}_1 \cdot c_3$, and the token of c_1 is suppressed when $\bar{c}_1 \cdot \bar{c}_3$. For c_7 (Figure 5.6b), both the outer **BRANCH** (select c_1) and the inner **BRANCH** (select c_3) require suppression. The expressions are derived by the same construction.

Applying Distribution to All Levels

The same splitting and distribution procedure applies to every level ROBDD. For instance, the level-1 ROBDD of the running example ($\{c_1, c_2, c_3, c_4\}$ in Figure 5.4d) has c_4 reachable by two paths and should be split into $c_{4,1}$ and $c_{4,2}$, with a distribution **BRANCH** on select c_1 .

Each level ROBDD is processed independently. The distribution circuit for a given level consumes and produces only the condition tokens belonging to that level, which have already been filtered by the higher-level circuits of Section 5.3. The splitting, routing, and select-side suppression therefore operate entirely within one level and do not affect the token balance between levels. The internal token-matching invariants of each level's circuit follow from the read-once arguments given above. The remaining question of how tokens are delivered across levels is addressed in the next section.

5.6 Token Demotion

Sections 5.4 and 5.5 constructed, for each level ROBDD, a read-once MUX-tree with distribution circuits that satisfy the token-matching invariants internally. These circuits operate

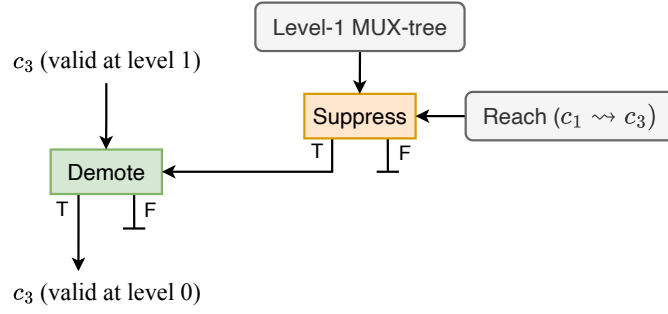


Figure 5.7: Demotion of c_3 from level 1 to level 0. The level-1 MUX-tree output is filtered by a suppression BRANCH before driving the demotion BRANCH.

within a single nesting level. The remaining task is to connect the levels.

A condition variable at nesting level k may produce a token on each iteration of the level- k loop, but the level-0 MUX-tree requires a single token per firing of B_p , corresponding to the variable's value on the final iteration of that loop. *Demotion* filters the tokens so that only the one from the final iteration reaches the lower level. At each level from k down to 1, a *demotion branch* receives the condition token on its data input and the output of that level's MUX-tree on its select input. When the MUX-tree output indicates that the loop exits, the token passes through and becomes valid at the next lower level. When the output indicates another iteration, the token is discarded. As established in Section 5.3, a variable that appears in a lower-level ROBDD also appears in every intermediate-level ROBDD, so it can always be demoted through successive levels. Not every variable requires demotion all the way to level 0. If a variable does not appear in the ROBDD at some level j , it does not appear at any level below j either, and demotion stops.

In the running example, consider c_1 , which belongs to level 1 and appears in the level-0 ROBDD. BB1 produces a c_1 token on every iteration of the outer loop. The level-1 MUX-tree evaluates the loop exit condition on each iteration. A demotion branch receives the c_1 token on its data input and the level-1 MUX-tree output on its select input. The token passes through when the loop exits and is discarded otherwise. Since c_1 is the root of the level-1 ROBDD, the MUX-tree evaluates precisely on the iterations where c_1 is produced, and the demotion branch's data and select inputs are both present on every iteration.

The situation is not always this simple. Consider c_3 , which belongs to level 2. Demotion through level 2 is handled in the same way as c_1 through level 1, because c_3 is the root of the level-2 ROBDD. After this step, the c_3 token is valid at level 1 and must be demoted to level 0. The level-1 ROBDD has root c_1 , and its MUX-tree produces an output on every iteration of the outer loop. But c_3 is reached only on iterations where c_1 is false. On other iterations, the MUX-tree output arrives at the demotion branch's select input with no c_3 token on the data input, and the select token is never consumed.

The solution is the same construction used for the select inputs of distribution branches in Section 5.5. A suppress component is inserted on the demotion branch's select input, driven by the reachability of c_3 from the root c_1 in the level-1 ROBDD. On iterations where c_3 is

Algorithm 6: Token Demotion**Input:** Level ROBDDs for levels $0, 1, \dots, K$, with split and distributed MUX-trees**Output:** Demotion circuits connecting all levels

```

1 foreach condition variable  $c_B$  at nesting level  $k > 0$  do
2   for  $j = k$  to 1 do
3     if  $c_B$  does not appear in the level- $(j-1)$  ROBDD then
4       break;
5     insert a demotion branch with;
6       data input:  $c_B$  token, valid at level  $j$ ;
7       select input: level- $j$  MUX-tree output;;
8       suppressed using the reachability of  $c_B$  from the root of the level- $j$ 
        ROBDD;
9       true output:  $c_B$  token, now valid at level  $j-1$ ;
10      false output: discarded;

```

not reached, the MUX-tree output is discarded before it reaches the demotion branch. On iterations where c_3 is reached, the output passes through as before. Figure 5.7 shows the demotion circuit for c_3 through level 1.

In general, for any variable c_B being demoted through level j , a suppress component is placed on the demotion branch's select input. The suppress condition is derived from the reachability of c_B from the root of the level- j ROBDD, using the same sub-ROBDD construction as in Section 5.5. When c_B is the root, the reachability condition is constant true, the suppress component is never activated, and it is removed during optimization. Algorithm 6 summarizes the procedure.

We verify that the demotion circuit satisfies the token-matching invariants. The MUX-trees within each level are internally token-matched by the read-once and distribution arguments of the preceding sections. It remains to check the BRANCH components. The suppression BRANCH on the demotion BRANCH's select input receives the level- j MUX-tree output on its data input and the reachability condition on its select input. Both are evaluated once per iteration of the level- j loop, so a token arrives on each input on every iteration and both are consumed together. After suppression, the demotion BRANCH's select input receives a token only on iterations where c_B is reached. This is exactly the condition under which c_B token arrives on the demotion BRANCH's data input. The two inputs are therefore always consumed together, and no token is left unconsumed on either side.

The demotion and distribution circuits both rely on BRANCH components to filter condition tokens. To combine them into a single suppression circuit, we must specify how a variable's token flows through both mechanisms without entangling their semantics.

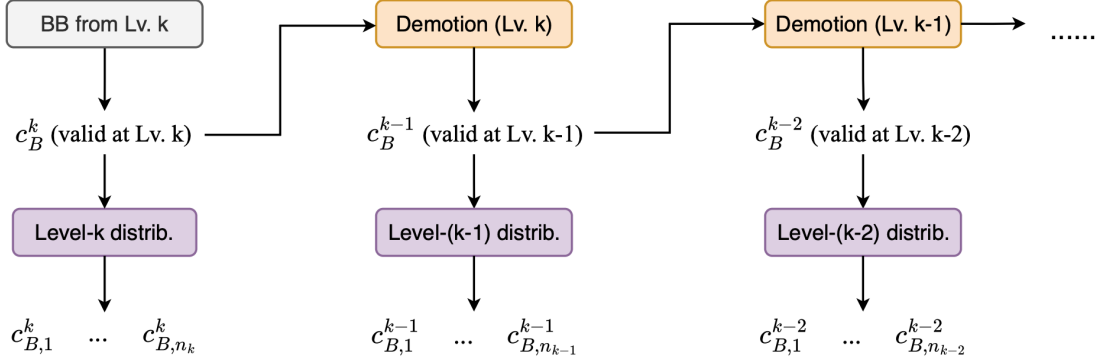


Figure 5.8: Token flow for a condition variable c_B at nesting level k . At each level, the token is forked: one copy enters the distribution circuit of that level, the other is demoted to the next lower level. Demotion and distribution are independent at each level.

Assembling the Suppression Circuit

Demotion operates on the original token produced by the basic block, not on the split copies created by distribution. Each level ROBDD has a different root and a different path structure, so the number of copies, their LCA vertices, and the distribution branch trees differ from level to level. A copy produced for one level's distribution may not correspond to any input of another level's MUX-tree and cannot be demoted in place of the original token. Demotion and distribution are therefore kept strictly separate, as shown in Figure 5.8. The basic block at level k produces a single c_B token, which is duplicated by a **FORK**. One copy enters the level- k distribution circuit, which routes it to the split inputs of the level- k MUX-tree. The other copy enters the demotion circuit. After filtering by the level- k demotion branch, the token becomes valid at level $k-1$ and is again duplicated by a **FORK**: one copy enters the level- $(k-1)$ distribution circuit, and the other continues to the next demotion stage. This process repeats until the token reaches the lowest level at which c_B appears.

This completes the suppression circuit for the case where the consumer consumes a token whenever its block is reached. Starting from the local CFG (Section 5.1), the decision graph (Section 5.2) and its level ROBDDs (Section 5.3) determine the Boolean expressions to be evaluated. Splitting (Section 5.4) and distribution (Section 5.5) produce read-once MUX-trees whose internal token-matching follows from the read-once property. Demotion (Section 5.6) connects the levels without disturbing the token-matching of any previously constructed circuit. Each mechanism adds **BRANCH** components whose token-matching is verified within the same section, and the complete circuit satisfies the token-matching invariants without requiring retroactive fixes. In the running example, applying this procedure to the CFG of Figure 5.4a produces a suppression circuit for the delivery from B_p to B_c , with level ROBDDs, split MUX-trees, distribution branches, and demotion branches as developed throughout this chapter.

The remaining case arises when the consumer is an input of a γ -tree and may not consume a token even when its block is reached. As we shall see, this invalidates the dominance

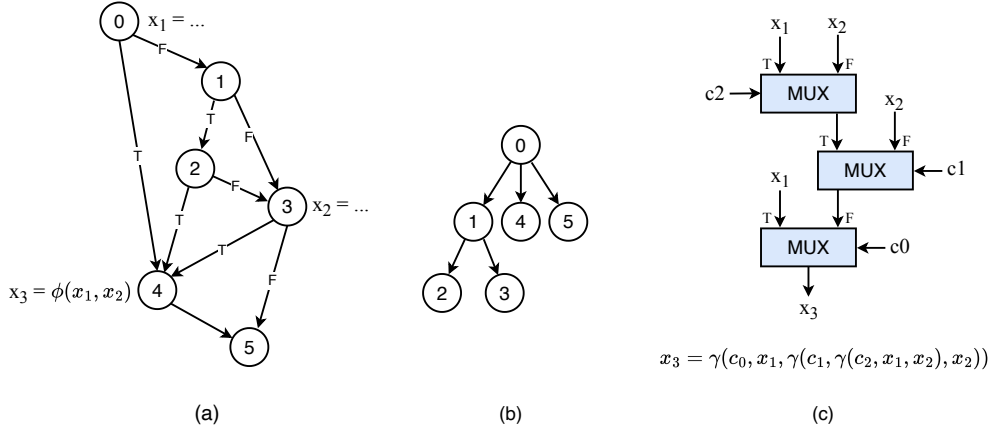


Figure 5.9: Consumer on a γ -tree. (a) CFG with two definitions of x : x_1 in BB0 and x_2 in BB3. (b) The dominator tree. (c) The γ -tree in BB4, selecting between x_1 and x_2 .

property on which the local CFG construction of Section 5.1 depends and requires a different starting point for the analysis.

5.7 Conditional Consumption

The suppression circuit construction of Sections 5.1–5.6 relies on the fact that B_p dominates every block whose condition variable appears in the suppression expression. This fact was established in Section 5.1 from a chain of reasoning that begins with the assumption that the consumer consumes a token whenever its block B_c is reached. Under this assumption, GSA guarantees that the producer's value is needed on every path to B_c , so B_p must dominate B_c .

When the consumer is an input of a γ -tree, the consumer's block B_c may be reached without the token being consumed, because the MUX may select a different input. The block B_c no longer requires B_p 's value on every path, since other producers in the γ -tree can supply the definition instead. Therefore B_p does not necessarily dominate B_c , and the suppression expression may involve condition variables from blocks that B_p does not dominate.

Figure 5.9 illustrates this situation. Variable x has two definitions, x_1 in BB0 and x_2 in BB3. Figure 5.9a shows the CFG, Figure 5.9b the dominator tree, and Figure 5.9c the γ -tree in BB4 that selects between the two definitions using three MUX components with select signals c_0 , c_1 , and c_2 .

Every input of the γ -tree, including the select signals, is the consumer of a delivery. Consider the delivery of the select signal c_2 from BB2 to BB4. Here $B_p = \text{BB2}$ and $B_c = \text{BB4}$. BB2 does not dominate BB4, because the path $\text{BB0} \rightarrow \text{BB4}$ reaches BB4 without passing through BB2. The suppression condition is $\overline{c_2} \cdot \overline{c_3}$, which involves c_3 from BB3. BB2 does not dominate BB3 either: the path $\text{BB0} \rightarrow \text{BB1} \rightarrow \text{BB3}$ reaches BB3 without passing through BB2. The c_3 token may therefore arrive at the suppression expression circuit with no c_2 token on the data input of the suppression BRANCH, violating the token-matching invariants. The

assumption that B_p dominates every condition variable in the suppression expression, which was the foundation of the correctness arguments in Sections 5.1–5.6, no longer holds.

The γ -tree introduces a further complication beyond this loss of dominance. Consider suppressing the delivery of x_2 from BB3 to its input on the MUX with select c_1 . Following [6], the suppression condition (and therefore the transitive closure of the control dependence) must include the select signals of every MUX on the path from the γ -tree root to this input, because even when BB4 is reached, the token is consumed only if each select along the path evaluates to the value that leads to this input. However, a select condition whose block is upstream of B_p can be omitted when only one branch from that block can reach B_p , because whenever the producer fires, control must have taken the branch that reaches B_p , and the select value is already determined.

In this example, the path from the root to the chosen x_2 input passes through c_0 and c_1 . For c_0 (BB0), one branch reaches BB3 and the other does not, so c_0 can be omitted. For c_1 (BB1), both branches can reach BB3, so c_1 cannot be omitted and must appear in the suppression expression. But BB3 does not dominate BB1. If c_1 is included in the suppression circuit, the c_1 token may arrive at the suppression BRANCH's select side with no token from the producer on its data input, violating the token-matching invariants.

The Dominator Block

To restore the dominance property on which the construction of Sections 5.1–5.6 depends, we move the starting point of the analysis from B_p to a block that dominates both B_p and every condition block appearing in the suppression expression. We call this block the *dominator block* B_d , defined as the LCA of B_p and these condition blocks in the dominator tree. The consumer block B_c is not included in the computation, since no token in the suppression circuit originates from B_c .

For the two deliveries examined above, B_d is computed as follows. For the delivery of c_2 , the condition blocks in the suppression expression are BB2 and BB3, and the LCA of these blocks together with $B_p = \text{BB2}$ in the dominator tree (Figure 5.9b) is BB1, so $B_d = \text{BB1}$. For the delivery of x_2 , the condition blocks are BB1 (select of MUX) and BB3 (reaching condition from BB3 to BB4), and the LCA of the condition blocks and $B_p = \text{BB3}$ is again BB1, so $B_d = \text{BB1}$. In both cases, B_d dominates B_p as well as every condition block in the expression.

Building the Expression from the Dominator Block

With B_d in hand, an intermediate goal is to construct an expression circuit that, starting from B_d , evaluates whether the execution reaches B_c and the consumer input on the γ -tree is consumed. We build this circuit in two steps. First, we apply the construction of Sections 5.1–5.6 with B_d as the root of the local CFG and B_c as the consumer block. The output is a circuit that produces a correct reaching condition expression from B_d to B_c . All the arguments of those sections carry over.

Second, we incorporate the γ -tree select conditions along the path from the root to the consumer input. Reaching B_c is not sufficient for the consumer input to be consumed: each γ -tree select on this path must take the value that leads to the input. The rule that

omits a select whose block is upstream with only one branch reaching the producer must be reevaluated with B_d as the reference rather than B_p , since the analysis now starts from B_d . The result is a Boolean expression that is true exactly when the execution from B_d reaches B_c and the consumer input is consumed.

Concretely, we modify the level-0 ROBDD. For each vertex in the split level-0 ROBDD whose variable corresponds to a γ -tree MUX select, we redirect the outgoing edge on the side that does not lead to the consumer input directly to the false terminal. The result is equivalent to pruning the corresponding MUX-tree.

Three points explain this construction.

Only the level-0 ROBDD is modified. A γ -tree select governs whether the consumer input is consumed, which is independent of whether a loop returns to its header or exits. A detail worth noting is that, before the modification, the level-0 ROBDD may not be strictly an ROBDD: both successors of a MUX select's block may be post-dominated by the block containing the γ -tree, so both outgoing edges of the corresponding vertex point to the true terminal, and only after one of them is redirected to the false terminal does the graph become an ROBDD. This is the only irregularity that MUX selects introduce into our construction. It does not affect distribution or demotion, but it affects the construction of the level-0 ROBDD: during the reduction, we must not optimize out a vertex corresponding to a MUX select. For convenience, we continue to refer to this intermediate object as the level-0 ROBDD.

Every relevant γ -tree select appears in the level-0 ROBDD. This ensures that every MUX select that should be incorporated in the expression can be found in the ROBDD and used in the modification. To prove it, a MUX select upstream of B_d that is not omitted is one whose block has both branches able to reach B_d , and therefore both branches reach B_p , placing the block in B_p 's condition set. But then B_d cannot dominate this upstream block, leading to a contradiction. Therefore, an upstream select of B_d must be omitted. For a MUX select whose block is downstream of B_d , it must appear in the local CFG. It is added to the transitive closure of the control dependence of B'_c , and therefore appears as a vertex in the decision graph. As noted above, the reduction during the level-0 ROBDD construction does not optimize out MUX select vertices, so the block appears in the level-0 ROBDD.

The modification is performed after distribution. Distribution is built from the split level-0 ROBDD before pruning takes place. Pruning may remove a subtree of the MUX-tree that contains one of several split copies of a variable, eliminating the MUX input to which that copy was routed. The corresponding output of the distribution branch tree then has no consumer and will be connected to a SINK. The remaining copies and their distribution are unaffected. If pruning were performed first, the split ROBDD would not reflect the original sharing structure, and the distribution branch trees could not be constructed correctly.

The expression computed so far captures whether, starting from B_d , the execution reaches B_c and the consumer input on the γ -tree is consumed. This is not yet the suppression

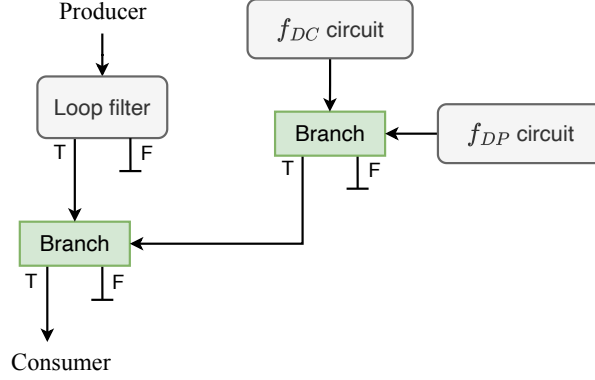


Figure 5.10: Suppression circuit for a γ -tree consumer. f_{DC} is the dominator-to-consumer expression and f_{DP} the dominator-to-producer expression. The f_{DC} output is filtered by f_{DP} to discard activations where the producer does not fire. The producer's token is filtered by a loop-filtering circuit to retain only the final iteration. The main **BRANCH** receives the two filtered streams.

condition we need. The suppression **BRANCH** receives its data token from B_p , not from B_d , so the expression output must be reconciled with the producer's token stream.

Circuit Structure

The suppression **BRANCH** has the producer's token on its data input and the expression output on its select input. For this **BRANCH** to satisfy the token-matching invariants, every token on one side must have a corresponding token on the other. We first show that neither side lacks a token when the consumer needs a token, then address the cases where extra tokens may arrive.

GSA already guarantees that whenever the consumer input is consumed, the producer has fired and a data token is available (Section 4.1). It remains to show that a token from the expression circuit is also available, which requires that the delivery from B_d to B_c does not cross a loop boundary from outside to inside. The γ -tree selects among definitions that are all at the same loop nesting level as the consumer, because any definition entering from an outer loop would be handled by a μ -function at the loop header rather than by the γ -tree. The loop header of this nesting level dominates every definition block and thus every condition block in the γ -tree. Since B_d is the LCA of B_p (which is either a definition or a γ -condition) and these condition blocks in the dominator tree, it cannot lie above the loop header, and is therefore at the same nesting level as the consumer.

Together, these facts mean that whenever the consumer input is consumed, an expression token and a producer token are available. The suppression **BRANCH** will therefore never lack a necessary token on either side.

The remaining concern is that extra tokens may arrive on one side without a corresponding token on the other. Two situations arise.

The first is that B_d is activated but B_p does not fire. In this case the expression circuit produces an output token, but no data token arrives at the main **BRANCH**. The solution uses the same construction applied to the select inputs of distribution branches in Section 5.5. A separate expression is computed using the full construction of Sections 5.1–5.6, with B_d as the starting block and B_p as the consumer. Since B_d dominates B_p , this is a standard non- γ delivery. A **BRANCH** with this expression on its select input and the dominator-to-consumer output on its data input discards the dominator-to-consumer token whenever B_p is not reached. When B_p does fire, the filtered token passes through to the main suppression **BRANCH**'s select input.

The second situation is that B_p lies inside a loop that does not contain B_d in the local CFG built from B_d . This is a Case 4 loop from Section 5.1: After reaching B_p from B_d , B_p may fire on every iteration, producing multiple data tokens per activation of B_d , while the filtered dominator-to-consumer expression token arrives only once. To solve this mismatch, a loop-filtering circuit is inserted before the main **BRANCH** to retain only the token from the final iteration and discard the rest. The construction is similar to the demotion mechanism of Section 5.6.

Figure 5.10 shows the complete circuit. The figure labels the dominator-to-consumer expression as f_{DC} and the dominator-to-producer expression as f_{DP} . The f_{DC} output is filtered by the f_{DP} **BRANCH**, the producer's token is filtered by the loop-filtering circuit, and the two filtered streams meet at the main suppression **BRANCH**.

This circuit structure subsumes the non- γ case of Sections 5.1–5.6. When the consumer is not on a γ -tree, B_d coincides with B_p , and B_p does not appear inside a Case 4 loop in its own local CFG. The dominator-to-producer expression is therefore constant true, the f_{DP} **BRANCH** and the loop-filtering circuit are both optimized away, and the circuit reduces to a single **BRANCH** with the expression output on its select input and the producer's token on its data input.

This completes the suppression algorithm. For any producer-consumer delivery, the circuit of Figure 5.10 evaluates whether the consumer will consume the token and discards it otherwise, with all internal token deliveries satisfying the token-matching invariants.

Experimental Results

We integrated the methodology developed in this thesis into Dynamatic [4], an open-source C-to-elastic-circuits compiler based on LLVM [21]. Specifically, we replaced the circuit generation pass of Dynamatic with a custom LLVM pass that implements the complete procedure described in Chapters 4 and 5: GSA conversion, regeneration, and the suppression algorithm including distribution, demotion, and the γ -tree extension. The back-end flow is unchanged: we use Dynamatic’s existing infrastructure to insert buffers using the MILP-based algorithm of Josipović et al. [11] and to generate the RTL description of the circuits.

We synthesize the generated VHDL netlists using Vivado [22] targeting a Xilinx Kintex-7 FPGA with a clock period constraint of 4 ns. We simulate the designs using ModelSim [23] and apply test vectors for functional verification. For each benchmark, we measure the cycle count from simulation, the clock period (CP) from post-routing timing analysis, the total execution time (cycles $\times$ CP), and the resource usage (LUT and FF counts) reported by Vivado after placement and routing.

To validate the robustness of our implementation, we evaluated it against Dynamatic’s official integration test suite, which consists of diverse kernels designed to exercise different control structures and trigger potential corner cases. Our implementation passes all 99 benchmarks that are fully functional and verifiable on the baseline framework, confirming that the formalized algorithm handles all control structures encountered in practice.

Table 6.1 and Figure 6.1 summarize the results of eleven benchmarks, comparing the circuits produced by our implementation (Ours) against those produced by the baseline Dynamatic [4], which uses the literal translation strategy of Josipović et al. [1].

The results demonstrate consistent improvements in both execution time and resource utilization across the majority of the benchmarks. Our approach reduces execution time in nine of eleven cases, with improvements ranging from 5% to 49%. LUT usage is reduced in ten cases and FF usage in nine. The largest execution time reduction is achieved by **Binary_search** (49%), and the largest area reduction by **Gcd** (32% fewer LUTs, 43% fewer FFs).

Benchmarks with deep loop nests, such as **Stencil_2d** and **Sobel**, benefit from the ability to overlap outer loop iterations with the execution of inner loops, a form of parallelism that the baseline’s in-order control network cannot exploit [6]. Benchmarks with simple iterative structures, such as **Gcd** and **Factorial**, show modest changes in execution time but significant area savings, because the direct delivery eliminates the redundant steering components that

Table 6.1: Elastic circuits produced by our methodology, contrasted to those produced by the open-source tool Dynamatic [4]. We measure cycle counts in simulation and obtain the timing and resources from Vivado, after place-and-route.

Benchmark	Cycle count		CP (ns)		Execution time (μ s)			LUTs			FFs		
	[4]	Ours	[4]	Ours	[4]	Ours		[4]	Ours		[4]	Ours	
Stencil_2d	652	513	4.38	3.92	2.86	2.01	-30%	1155	984	-15%	801	854	+7%
Binary_search	238	156	5.07	3.96	1.21	0.62	-49%	963	810	-16%	873	653	-25%
Sobel	5151	3526	5.09	4.91	26.19	17.31	-34%	3423	2919	-15%	3042	2662	-12%
Matvec	20107	20008	3.76	3.87	75.60	77.43	+2%	774	740	-4%	726	645	-11%
Gemm	46229	46426	7.19	6.57	332.39	305.02	-8%	3635	3348	-8%	2417	2380	-2%
Kmp	4600	3748	8.20	7.37	37.72	27.62	-27%	11628	11617	0%	3656	3630	-1%
Spmv	78	54	5.07	4.39	0.40	0.24	-40%	2018	1384	-31%	1857	1218	-34%
Gcd	151	163	4.44	4.15	0.67	0.68	+1%	2717	1849	-32%	2336	1329	-43%
Factorial	131	131	4.14	3.86	0.54	0.51	-6%	986	879	-11%	846	798	-6%
Iterative_div	7	7	4.58	4.25	0.03	0.03	-7%	945	992	+5%	657	708	+8%
Mvt_float	10567	10220	7.37	7.27	77.88	74.30	-5%	3669	3621	-1%	2156	2124	-1%

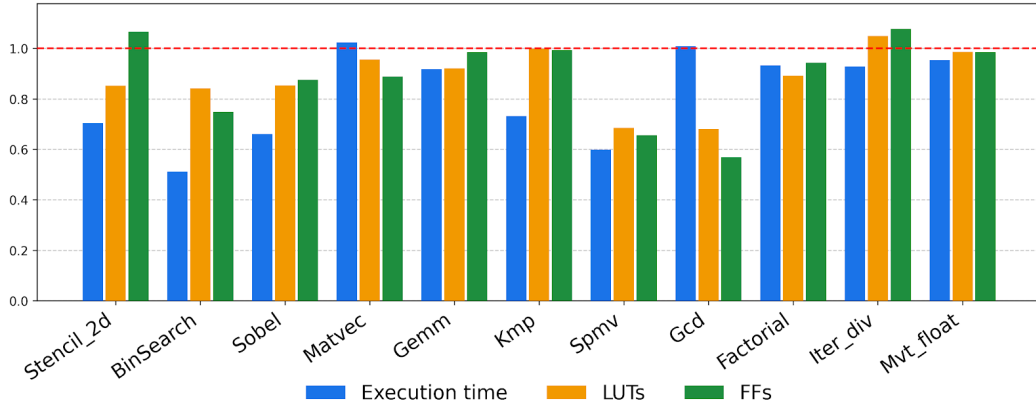


Figure 6.1: Results. Execution time and resources of our circuits, normalized to the results of Dynamatic [4].

the literal translation inserts between basic blocks. Two benchmarks show slight regressions on individual metrics: **Matvec** has a 2% increase in execution time despite fewer cycles, due to a marginally longer clock period, and **Iterative_div** uses slightly more LUTs and FFs.

Overall, the results confirm that the formalized algorithm of this thesis preserves the performance and area advantages of the Fast Token Delivery methodology [6] while providing the robustness guarantees developed in Chapter 5.

Conclusion

This thesis presented a complete suppression algorithm for the Fast Token Delivery methodology. The central difficulty addressed by the algorithm is that the suppression circuit introduces internal token deliveries that cannot be resolved by the original delivery procedure. Our algorithm handles these deliveries during construction through a sequence of steps: the control structure relevant to a producer-consumer pair is extracted into a local CFG and reduced to a decision graph, which is decomposed into per-loop ROBDDs. Each ROBDD is mapped to a read-once MUX-tree via a splitting transformation, and distribution branch trees route the original tokens to the correct split inputs. When the decision graph contains cycles, demotion branches filter condition tokens across nesting levels. Each step preserves the token-matching invariants established by the preceding steps. The algorithm is extended to handle consumers on γ -trees, where the loss of dominance between producer and consumer requires a modified starting point for the analysis.

The implementation within Dynamatic passes all 99 integration tests and produces circuits with significant improvements in execution time and area over the baseline. The results confirm that the performance advantages reported by Elakhras et al. for Fast Token Delivery are preserved under the complete algorithm.

We are currently exploring a generalization of the ROBDD-to-circuit mapping that replaces Shannon expansion with a broader class of decompositions. In the current approach, each MUX in the tree has a single condition variable on its select input, corresponding to Shannon expansion at the root vertex of a sub-ROBDD. More generally, a MUX's select input can itself be a subtree. Given an ROBDD, if two vertices can be identified such that every root-to-terminal path passes through one of them, the ROBDD decomposes into the subgraph from the root to these two vertices, which becomes the select subtree, and the two subgraphs from these vertices to the terminals, which become the two data inputs. Shannon expansion is a special case of this decomposition, where the root vertex alone serves as the select and its two children induce the two data subgraphs. Other decompositions can reduce the number of times a variable appears across the MUX-tree inputs, making the read-once property easier to achieve and reducing the number of distribution circuits required. This has the potential to improve both area and timing of the generated circuits. We have developed algorithms for such decompositions and established that it can be integrated into the Fast Token Delivery procedure of this thesis. The implementation and experimental evaluation of the algorithms are left as future work.

Bibliography

- [1] L. Josipović, R. Ghosal, and P. Ienne, “Dynamically scheduled high-level synthesis,” in *Proceedings of the 26th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, Monterey, CA, Feb. 2018, pp. 127–36.
- [2] M. Budiu, P. V. Artigas, and S. C. Goldstein, “Dataflow: A complement to superscalar,” in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*, Austin, TX, Mar. 2005, pp. 177–86.
- [3] L. Josipović, A. Guerrieri, and P. Ienne, “Synthesizing general-purpose code into dynamically scheduled circuits,” *IEEE Circuits and Systems Magazine*, vol. 21, no. 1, pp. 97–118, May 2021.
- [4] “Dynamatic,” 2026. [Online]. Available: <https://github.com/EPFL-LAP/dynamatic>
- [5] L. Josipović, A. Guerrieri, and P. Ienne, “Dynamatic: From C/C++ to dynamically scheduled circuits,” in *Proceedings of the 28th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, Seaside, CA, Feb. 2020, pp. 1–10.
- [6] A. Elakhras, A. Guerrieri, L. Josipović, and P. Ienne, “Unleashing parallelism in elastic circuits with faster token delivery,” in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2022, pp. 253–261.
- [7] J. Ferrante, K. J. Ottenstein, and J. D. Warren, “The program dependence graph and its use in optimization,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 9, no. 3, pp. 319–349, 1987.
- [8] K. J. Ottenstein, R. A. Ballance, and A. B. MacCabe, “The program dependence web: a representation supporting control-, data-, and demand-driven interpretation of imperative languages,” in *Proceedings of the ACM SIGPLAN 1990 conference on Programming language design and implementation*, 1990, pp. 257–271.
- [9] P. L. Campbell, K. Krishna, and R. A. Ballance, “Refining and defining the program dependence web,” Sandia National Labs., Albuquerque, NM (United States), Tech. Rep., 1993.
- [10] J. Cortadella, M. Kishinevsky, and B. Grundmann, “Synthesis of synchronous elastic architectures,” in *Proceedings of the 43rd Design Automation Conference*, San Francisco, CA, Jul. 2006, pp. 657–62.
- [11] L. Josipović, S. Sheikhha, A. Guerrieri, P. Ienne, and J. Cortadella, “Buffer placement and sizing for high-performance dataflow circuits,” in *Proceedings of the 28th*

- ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, Seaside, CA, Feb. 2020, pp. 186–96.
- [12] K. D. Cooper and L. Torczon, *Engineering a compiler*. Morgan Kaufmann, 2022.
- [13] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 13, no. 4, pp. 451–490, 1991.
- [14] R. E. Bryant, “Graph-based algorithms for boolean function manipulation,” *Computers, IEEE Transactions on*, vol. 100, no. 8, pp. 677–691, 1986.
- [15] K. S. Brace, R. L. Rudell, and R. E. Bryant, “Efficient implementation of a bdd package,” in *Proceedings of the 27th ACM/IEEE design automation conference*, 1991, pp. 40–45.
- [16] F. E. Allen, “Control flow analysis,” *ACM Sigplan Notices*, vol. 5, no. 7, pp. 1–19, 1970.
- [17] M. S. Hecht and J. D. Ullman, “Flow graph reducibility,” in *Proceedings of the fourth annual ACM symposium on Theory of computing*, 1972, pp. 238–250.
- [18] R. Tarjan, “Testing flow graph reducibility,” in *Proceedings of the fifth annual ACM symposium on Theory of computing*, 1973, pp. 96–107.
- [19] M. Karchmer, N. Linial, I. Newman, M. Saks, and A. Wigderson, “Combinatorial characterization of read-once formulae,” *Discrete Mathematics*, vol. 114, no. 1-3, pp. 275–282, 1993.
- [20] N. H. Bshouty, T. R. Hancock, and L. Hellerstein, “Learning boolean read-once formulas over generalized bases,” *Journal of Computer and System Sciences*, vol. 50, no. 3, pp. 521–542, 1995.
- [21] <http://www.llvm.org>, The LLVM Compiler Infrastructure, 2018. [Online]. Available: <http://www.llvm.org>
- [22] *Vivado Design Suite*, Xilinx Inc., 2020. [Online]. Available: <https://docs.xilinx.com/v/u/2019.2-English/ug901-vivado-synthesis>
- [23] Mentor Graphics, “ModelSim,” 2016. [Online]. Available: <https://www.mentor.com/products/fv/modelsim/>